University of Tübingen
Faculty of Science
Department of Computer Science

Master's Thesis in Machine Learning

Neural ODEs and Parameter Inference in Differentiable Neuron Simulations

Dóra Viktória Molnár

2025

Supervisor:

Prof. Dr. Jakob H. Macke
Machine Learning in Science
Department of Computer Science
University of Tübingen

Reviewer:

Prof. Dr. Philipp Hennig
Methods of Machine Learning
Department of Computer Science
University of Tübingen

Molnár, Dóra Viktória:
Neural ODEs and Parameter Inference in Differentiable Neuron Simulations
Master's Thesis in Machine Learning
University of Tübingen
Tübingen, April – October 2025

Abstract

Biophysical neuron models have long been used to link experimental data with theoretical understanding of neural dynamics, but they often fall short of reproducing the full complexity of recorded voltage traces. This motivates the search for more expressive models that are able to reproduce voltage responses as faithfully as possible. This thesis explores how neural ordinary differential equations (neural ODEs) can address this problem by providing a flexible, data-driven extension to classical modeling. In this framework, hidden states evolve according to neural dynamics governed by ODEs and generate additional input currents, enabling compact single-compartment models to capture behaviors that would otherwise require detailed morphologies. Applied to passive neurons, this approach allows single-compartment setups to approximate multi-compartment simulations with high precision while remaining computationally efficient. Extending the framework to active models and experimental recordings introduces new challenges, particularly in handling spike dynamics and learning intrinsic parameters. The method addresses these by jointly learning a neural ODE alongside parameters such as ion channel conductances, compartment length, and membrane properties. This dual capability increases interpretability and makes it possible to test robustness across different cells and recording conditions. Longer traces and recordings with varying spike characteristics further demonstrate that the neural ODE system helps maintain generalization and delivers meaningful improvements under demanding scenarios. Finally, I experiment with alternative channel types, including Pospischil-style mechanisms, and explore more flexible channel dynamics by making the opening and closing kinetics of ion channels trainable. These experiments reveal that more flexible descriptions of ion channels can enhance realism, yet the neural ODE augmentation remains central to capturing dynamics beyond the reach of standard models. Altogether, the results highlight neural ODEs as a powerful framework that balances biological plausibility with computational tractability, offering a versatile path toward more expressive and data-aligned biophysical models.

Acknowledgements

I would like to express my warmest gratitude to my supervisor, Michael Deistler, for his invaluable guidance, encouragement, and infinite patience he provided throughout the course of this thesis. His expertise and support have been crucial for shaping both my research and my academic development.

I am also grateful to the members of the MackeLab for their insightful discussions, feedback, and for creating a collaborative and inspiring working environment.

I would like to thank my family and friends for their unconditional support and for always believing in me, even when I could not. Their constant encouragement has been the foundation that made this work possible.

Finally, I acknowledge the financial support provided by the ELIZA Master's Scholarship, which made it possible for me to pursue this degree and carry out my research.

Table of Contents

1 Introduction

Understanding how neurons respond to different stimuli is a fundamental goal in neuroscience. Computational biophysical models play a crucial role in this process by providing a controlled and interpretable framework for simulating neural behavior. These models, based on systems of ordinary differential equations (ODEs), provide a mechanistic link between cellular properties and observed voltage traces (Almog and Korngreen, 2016). In particular, they describe how ion channel kinetics, membrane capacitance, and geometric features interact to produce the time evolution of the membrane potential. By recreating the electrical activity of neurons with the help of computational models, researchers can test hypotheses, study the effects of specific biophysical mechanisms, and predict how neurons might respond to new conditions or perturbations. These models also enable us to bridge the gap between experimental data and theoretical understanding, offering insight into the underlying dynamics of complex biological systems. As experimental techniques evolve, producing increasingly detailed recordings, there is a growing need for models that can both reproduce and explain these data faithfully and capture the richness of neural responses. To this end, in practice, we observe a voltage trace x_o from an experiment and aim to identify a model—or rather its parameters—that can reproduce it as closely as possible. Achieving this is important not only for capturing the behavior of individual cells, but also for building interpretable models that can generalize across experimental conditions.

Accurately inferring parameters in computational neuroscience models is a key step towards understanding neural dynamics and optimizing simulations. Biophysical neuron models—such as Hodgkin–Huxley-type models (Hodgkin and Huxley, 1952)—describe how membrane potentials evolve over time in response to synaptic inputs, ion channel kinetics, and intracellular processes. These models are typically governed by systems of differential equations parameterized by biological quantities, including ion channel conductances, membrane capacitance, and morphological properties. To faithfully reproduce the behavior of real neurons, these parameters must be estimated from the observed x_o. However, despite decades of progress, the task of fitting neuron models to data remains extremely challenging (Gerstner et al., 2014). One difficulty lies in the complexity of the underlying dynamics: nonlinear interactions between ion channels can produce sharp transitions such as action potentials, which are highly sensitive to parameter values. Even small deviations in conductances or time constants can lead to large mismatches between model output and x_o. Another obstacle is the limited and indirect nature of the data. Voltage recordings contain only the membrane potential, leaving other relevant variables such as gating variables or synaptic currents unobserved. This leads to problems of identifiability, where multiple parameter settings can produce similar outputs, making inference ill-posed (Bhalla and Bower, 1993). Moreover, biophysical models are often simplified to remain computationally tractable, for example, by collapsing dendritic trees into a single compartment. Such simplifications inevitably discard dynamical features of the real neuron, creating systematic mismatches that cannot be resolved by parameter tuning alone. In addition, the optimization landscape of these models poses a serious obstacle: the parameter space is high-dimensional and strongly non-convex, often containing several local minima and flat regions. Gradient-based methods may converge slowly or get trapped, while exhaustive search over parameters quickly becomes computationally infeasible. Furthermore, fitting models to diverse datasets, such as different stimulation methods or varying cell morphologies, adds to the complexity. These challenges have motivated a wide range of approaches, including statistical methods for smoothing and efficient estimation from noisy recordings (Huys and Paninski, 2009; Huys et al., 2006), evolutionary and multi-objective optimization frameworks such as BluePyOpt (Van Geit et al., 2016), and more recent simulation-based Bayesian inference strategies (Lueckmann et al., 2017).

In recent years, differentiable simulators have opened new directions for parameter inference and the training of mechanistic models. In computational neuroscience, Jaxley (Deistler et al., 2024) allows gradient-based optimization of neural models by leveraging automatic differentiation. These modern tools make it possible to explore more expressive and flexible inference strategies, including the use of neural networks to approximate unknown mechanisms or correct model discrepancies. In this work, we propose a novel application of neural ordinary differential equations (neural ODEs) to address the gap between simplified computational models and real neurons—an approach, to the best of my knowledge, that has not been explored before. A neural ODE augments classical ODE systems with flexible, data-driven components parameterized by neural networks. Instead of explicitly determining the entire right-hand side of the dynamics, the neural network can model hidden states that evolve alongside the biophysical variables. This makes it possible to capture dynamics that are missing from the base model (i.e., the biophysical model without neural ODE augmentation) while still retaining interpretability. Within this

framework, the base neuron model provides a biophysically grounded structure, while the neural ODE extension accounts for systematic discrepancies between the model output and the observation x_o. This hybrid design makes neural ODEs particularly well suited for fitting problems in neuroscience, where both interpretability and flexibility are essential.

In this thesis, I explore such a neural ODE-augmented system in a three-stage project.[1] The central idea is to use a neuron model as a base—first a computationally efficient but spatially simple single-compartment model, and later a more detailed multi-compartment one—and extend it with a neural ODE component that brings its output closer to target recordings denoted by x_o. In the beginning, x_o comes from multi-compartment simulations (serving as synthetic ground truth), while in later stages it originates from real experimental observations. The neural ODE system consists of two neural networks: one that evolves a latent state variable based on voltage input, and another that reads out an additional input current. This corrective current complements the original stimulus and enables the base model to approximate more realistic dynamics.

In the first part of the project, I focus on passive neuron models, which do not include active ion channels and therefore have simpler dynamics. This stage serves as a proof of concept: I test whether the neural ODE system can compensate for the structural gap between single-compartment models, which treat the neuron as a single electrical unit, and multi-compartment models, which divide the neuron into spatial segments. The results show that even when the base model lacks realistic structure, the augmented system can still recover important features of the target trace, such as the timing and amplitude of voltage changes.

The second part of this work tackles a more realistic and challenging setting: fitting an active single-compartment model to real experimental observations. Here, the goal is not only to correct mismatches in the model's output but also to learn its intrinsic biophysical parameters, such as ion channel densities, membrane properties, and compartment length, through gradient-based optimization. This parallel learning task increases both the difficulty and the expressiveness of the setup, since it combines interpretable parameter inference with data-driven corrective input. I begin with short voltage traces containing single spikes and progressively scale up to longer recordings. In addition, I experiment with Pospischil-type channels (Pospischil et al., 2008) as well as a modified kinetics formulation, where instead of using the fixed channel gating functions, I add small learnable, voltage-dependent offsets. This allows channel dynamics to be adjusted flexibly, while keeping the overall framework biophysically interpretable. Although these modifications yielded promising results, the neural ODE augmentation remained essential for achieving good fits.

As a third and final part of the project, I shift to a multi-compartment model, which more closely reflects the anatomical and electrical structure of real neurons by dividing them into interconnected segments. As before, the aim is to learn both the model parameters and the additive input current in parallel, but this time in a much more complex setting. By applying the same optimization framework to a structurally detailed model, I aim to assess how much morphological realism contributes to improved fits compared to the single-compartment case.

Across these three stages, I also investigate the stability of the latent dynamics, the effectiveness of different loss functions (e.g., Dynamic Time Warping or Sum of Squared Errors), and initialization strategies inspired by recent literature (Westny et al., 2024). I also experiment with training separate networks for spiking and subthreshold regimes, computing the optimal additive current that would give a perfect fit, exploring the distribution of the current across channels, and several other explorations that together provide a broader picture of the model's capabilities.

Overall, this work presents a hybrid approach to neuron modeling that integrates differentiable simulation, interpretable parameter optimization, and neural ODE–based augmentation. The resulting models remain grounded in biophysical principles while gaining the flexibility needed to closely match experimental voltage recordings. This methodology offers a promising direction for data-driven neuroscience applications, particularly in constructing personalized or experimentally constrained neuron models.

[1] To ensure reproducibility, all code developed for this thesis is publicly available at `https://github.com/dorimolnar/voltagefit_neural_ode.git`.

2 Background and Related Work

2.1 Neuron Modeling

Computational models of neurons are essential tools in neuroscience, allowing researchers to study and predict the electrical behavior of neurons under various conditions (Rattay et al., 2002). These models are typically formulated as systems of differential equations that describe how the membrane voltage evolves in response to synaptic inputs and intrinsic ionic currents. Two main aspects determine the complexity of a neuron model: its morphological structure and the type of ion channel dynamics it incorporates.

One fundamental distinction, that is also present in my work, is between single-compartment and multi-compartment models. In single-compartment models, the neuron is represented as a uniform point, assuming that the voltage is constant across the entire cell. These models are computationally efficient and analytically tractable, making them useful for large-scale simulations or parameter inference (Brette, 2015). However, they cannot capture spatial effects such as dendritic processing or the impact of the neuron's morphology on signal propagation.

In contrast, multi-compartment models divide the neuron into segments representing different anatomical parts (soma, dendrites, axon), connected via cable equations (Yang et al., 2020). These models can more accurately capture the spatial distribution of voltage and current flow, and are often used when precise reconstruction of a neuron's structure is available. However, they are more computationally intensive and require a greater number of parameters, many of which may be difficult to identify from limited experimental data.

A second important difference is between passive and active models (Georgiev, 2004). Passive models describe neurons using only passive membrane properties such as membrane resistance, capacitance, and leak conductance. They do not include voltage-gated ion channels and therefore cannot produce action potentials. Passive models are often used to study subthreshold dynamics or to initialize more complex models.

Active models, in contrast, incorporate nonlinear ionic currents, typically modeled using variants of the Hodgkin–Huxley formalism (Hodgkin and Huxley, 1952). These models include voltage- and time-dependent conductances for channels such as sodium, potassium, and calcium, allowing them to reproduce spiking behavior. While this setup is more biophysically accurate, active models are also more sensitive to parameter values and can be notoriously hard to fit to experimental data.

Various software tools have been developed to support the simulation of neuron models, including NEURON (Hines and Carnevale, 1997), Brian (Goodman and Brette, 2009), and more recently Jaxley (Deistler et al., 2024). Jaxley is a differentiable simulation library for conductance-based neuron models, implemented in JAX. It allows users to define both single- and multi-compartment, passive and active membrane models, simulate voltage traces, and compute gradients of outputs with respect to parameters. This makes it especially suitable for optimization and model-fitting tasks, while maintaining compatibility with high-performance machine learning infrastructure.

In this thesis, both passive and active single-compartment models are used. The passive model serves as a simplified baseline and is used to test the performance of the neural ODE framework in a controlled setting, including comparison to a passive multi-compartment model. The active model is used to fit real experimental voltage recordings that include spiking behavior, presenting additional challenges in terms of model stability and fitting accuracy. The goal is to explore whether neural ODE-based augmentation can enable a simple single-compartment simulation to reproduce the dynamics observed in more complex models or biological recordings.

2.2 Evaluating Neuron Models

A central goal of computational neuroscience is to develop neuron models that can accurately reproduce experimental voltage recordings given the same stimulation. However, when the predicted voltage trace is not an exact match, the questions arise: What level of accuracy is "good enough"? And when is one model "better" than another? At first glance, these may seem like simple issues, but defining meaningful evaluation criteria is surprisingly complex.

Jolivet et al. (2008) address this by exploring two fundamental questions: what makes a good neuron model, and how should we evaluate them? While comparing spike times is relatively straightforward,

evaluating the predicted membrane voltage is far more challenging. Traditional approaches like computing the squared error between the predicted and observed voltage can be misleading—especially around spikes, where even a small timing error can cause disproportionately large errors. At the same time, subtle differences in spike shape or the afterpotential can be biologically meaningful, especially for distinguishing neuron types.

To deal with this, the authors designed more nuanced evaluation metrics that combine various aspects of neural behavior, such as spike timing, subthreshold voltage accuracy, and spike-frequency adaptation. Crucially, they proposed normalizing model performance by the intrinsic reliability of the experimental data—i.e., how consistent the neuron itself is across repeated trials. This allows for a more interpretable assessment: rather than simply stating that a model has an error of 0.90, one can say how many standard deviations away the model is from the neuron's natural variability.

Beyond traditional metrics, recent advancements have introduced more sophisticated error functions. One such example is the Soft-DTW loss, which provides a differentiable approximation of Dynamic Time Warping (DTW). This approach enables alignment of time series with varying lengths and speeds, making it a valuable tool for training models on electrophysiological data (Cuturi and Blondel, 2018).

Another complementary evaluation method focuses on the similarity of entire spike trains rather than individual spikes. Metrics such as the Victor-Purpura distance (Victor and Purpura, 1997) or the van Rossum distance (Rossum, 2001) quantify differences in spike timing and patterns. These metrics are robust to small misalignments and provide a more global assessment of how well the temporal structure of the spiking activity is captured by the model. In addition to these, a wide variety of other metrics exist to evaluate different aspects of neural behavior, including subthreshold voltage dynamics, firing rate distributions, spike-frequency adaptation, and information-theoretic measures, reflecting the rich diversity of neural activity and the many ways models can be assessed (Brette and Gerstner, 2005; Paninski et al., 2007; Rall, 1962).

In summary, evaluating neuron models goes far beyond minimizing voltage error. It requires thoughtful criteria that reflect both biological significance and modeling goals. As Jolivet et al. (2008) emphasize, developing such criteria remains an open challenge in computational neuroscience.

2.3 Model Fitting and Parameter Inference

Fitting computational neuron models to experimental or simulated data involves adjusting model parameters so that the predicted membrane voltage closely matches x_o under the same stimulation. Several approaches exist: some rely on probabilistic or sampling-based methods, such as Bayesian inference, while others explicitly optimize a loss function that measures the mismatch between simulated and target traces (Druckmann et al., 2008; Van Geit et al., 2007). In this thesis, I adopt the latter perspective, treating model fitting as an optimization problem. Depending on the modeling goal, the loss function can emphasize spike timing, amplitude, subthreshold dynamics, or full-trace similarity, as discussed in Section 2.2.

Recent advances in automatic differentiation and differentiable simulators such as Jaxley have enabled the use of gradient-based optimization for parameter inference in biophysical models (Bradbury et al., 2018; Maclaurin et al., 2015). This means that gradients of the loss function with respect to model parameters (such as ion channel conductances, reversal potentials, or morphological properties) can be computed efficiently and used to update parameters via standard gradient-based algorithms, including variants like Adam (Kingma and Ba, 2017). Compared to traditional approaches relying on grid search, evolutionary algorithms, or manual tuning (Bhalla and Bower, 1993; Van Geit et al., 2016), this allows for more scalable and fine-grained inference.

Gradient-based methods for parameter inference in dynamical systems typically rely on backpropagation through time (Werbos, 2002) or adjoint sensitivity analysis (Pontryagin, 2018). These approaches make it possible to compute gradients of a loss function with respect to model parameters by differentiating through the system's dynamics and have therefore become central to modern differentiable simulators. In principle, such methods enable highly efficient optimization when combined with algorithms such as stochastic gradient descent, Adam, or more advanced adaptive optimizers.

Despite these advances, applying gradient-based optimization to spiking neuron models remains particularly challenging (Huh and Sejnowski, 2018). The nonlinear dynamics of conductance-based models give rise to sharp transitions near spike initiation, which can amplify numerical instabilities. This often results in exploding or vanishing gradients, poor convergence, or sensitivity to initialization. Moreover,

parameter degeneracy—when multiple parameter combinations yield similar voltage traces—further complicates inference and may lead to biologically implausible solutions even when the fit to the data is good (Nogaret, 2022).

A further source of difficulty arises when the base model structure itself is insufficient to reproduce the observed behavior. As Gutenkunst et al. (2007) argue, in systems that are not fully understood, it is often more effective to build models that are incomplete, tentative, and falsifiable, rather than attempting to precisely determine every parameter. In practice, no amount of parameter tuning can fully compensate for missing channel types, oversimplified gating kinetics, or restrictive morphological constraints. Parameter inference alone is therefore insufficient to bridge the gap between model output and data. This motivates the development of model extensions that increase the expressivity of the system while preserving interpretability. In the context of differentiable simulations, one promising approach is to augment biophysical models with additional learnable dynamics formulated as ordinary differential equations (Chen et al., 2018), which will be discussed in the next subsection.

2.4 Neural Ordinary Differential Equations

Recent advances in deep learning have highlighted the power of composing iterative transformations to represent complex dynamics. As Chen et al. (2018) describe, models such as residual networks, recurrent neural network decoders, and normalizing flows achieve this by updating a hidden state h_t through discrete steps:

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t),$$

where $t \in \{0, \dots, T\}$ and $h_t \in \mathbb{R}^D$. These updates can be interpreted as an Euler discretization of a continuous transformation (Lu et al., 2018; Ruthotto and Haber, 2020).

By adding more layers and taking smaller steps, we can consider the limit, where one can describe the evolution of the hidden state continuously in time using an ordinary differential equation (ODE) parameterized by a neural network:

$$\frac{\mathrm{d}\mathbf{h}(t)}{\mathrm{d}t} = f_\theta(\mathbf{h}(t), t),$$

where f_θ is the neural network with learnable parameters θ.

This basic formulation of neural ODEs (Chen et al., 2018; Rubanova et al., 2019) allows the hidden state to evolve smoothly over continuous time and enables gradient-based optimization through adjoint sensitivity (Pontryagin, 2018) or backpropagation through time (Werbos, 2002). These methods make it possible to propagate gradients of a loss function with respect to the neural network parameters through the ODE solver. In practice, this enables the use of modern gradient-based optimizers such as Adam (Kingma and Ba, 2017) to fit the augmented neuron model to voltage traces or other electrophysiological measurements (Rackauckas et al., 2020). In this work, I use neural ODEs to augment single- and multi-compartment neuron models, providing additional flexibility to capture unmodeled dynamics while jointly learning biophysical parameters.

The neural ODE framework is particularly appealing in the context of biophysical neuron modeling. By augmenting a base neuron model with additional neural ODE dynamics, one can compensate for structural limitations of the model while preserving interpretability of the core biophysical parameters (Lei and Mirams, 2021). For instance, channels that are omitted from a simplified compartmental model or subtle nonlinear interactions that are hard to capture with standard Hodgkin–Huxley-type equations could be effectively approximated by a neural network embedded in the ODE system.

Despite these advantages, incorporating neural ODEs into spiking neuron models is nontrivial. First, it is important to note that neural ODEs are not universal approximators: not every function can be represented within this framework, and their expressivity is fundamentally limited (Dupont et al., 2019). The stiff and highly nonlinear dynamics of action potential generation also pose challenges for numerical integration and gradient computation (Huh and Sejnowski, 2018). Moreover, care must be taken to prevent the neural network from producing unrealistic membrane dynamics or destabilizing the base model. Techniques such as gradient clipping, spectral normalization, or explicitly constraining the network output to physiologically plausible ranges are often necessary (Finlay et al., 2020; White et al., 2023).

Neural ODEs also provide a method for exploring model expressivity. By combining interpretable biophysical parameters with learnable neural ODE augmentation, it becomes possible to systematically study which features of the neuron model are essential to capture observed dynamics and which can

be compensated for by the neural network. This hybrid approach leverages the strengths of mechanistic modeling while benefiting from the universal function approximation capabilities of neural networks (Hornik et al., 1989), offering a principled route to more expressive yet interpretable neuron models.

In summary, neural ODEs extend traditional neuron modeling by providing a differentiable, continuous-time augmentation that can improve fit to experimental data, capture missing dynamics, and maintain interpretability of the underlying biophysical parameters. In the following chapters, I will describe how this framework is applied in my experiments to augment single- and multi-compartment neuron models incorporating different channel types.

3 Experimental Data

The data that I've been working with during this thesis, were obtained from the Allen Brain Atlas – Cell Types Database, a comprehensive resource cataloging mammalian (originating from humans and mice) brain cells through detailed electrophysiological recordings, morphological reconstructions, and transcriptomic profiling. This database is part of a multi-year project aimed at creating a census of cell types within the mammalian cerebral cortex.

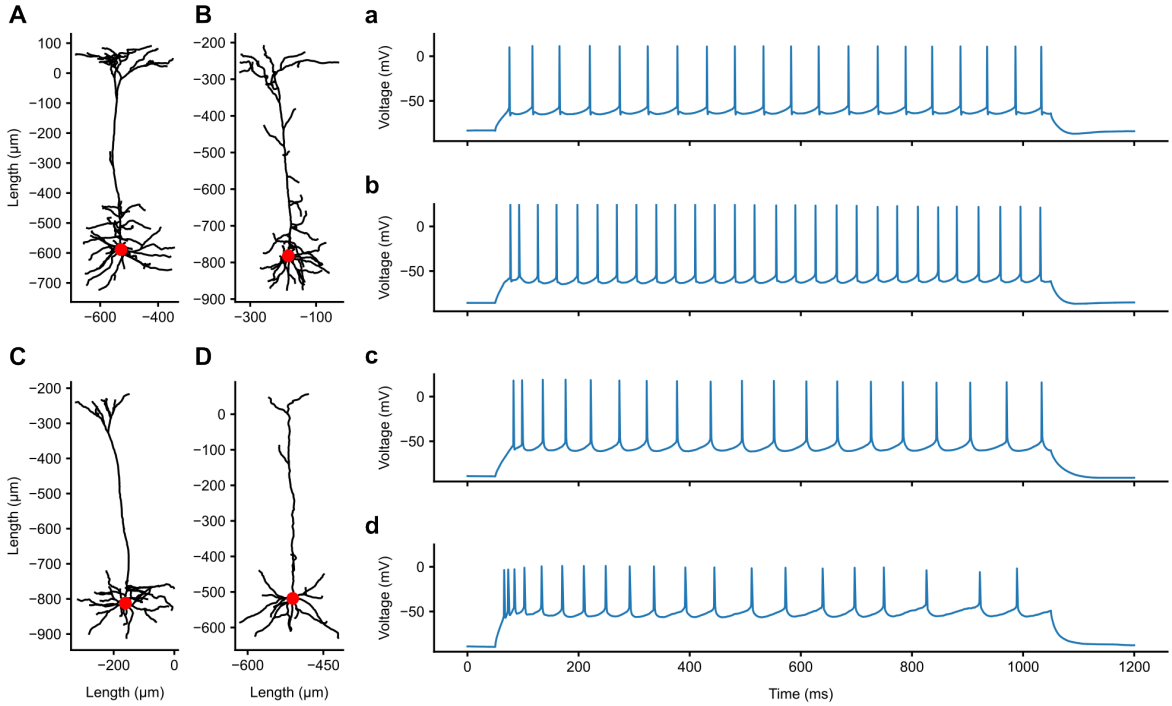

Figure 1: Morphologies and voltage responses of the four neurons used in this work. **Panels A–D** show the reconstructed morphologies of the selected neurons (with IDs 488683425, 485574832, 480353286, and 473601979) from the Allen Brain Atlas – Cell Types Database, with the soma highlighted in red. The axes show spatial coordinates in micrometers (µm). All cells are from the mouse primary visual cortex but exhibit diverse structural characteristics. **Panels a–d** display the corresponding somatic voltage responses of these cells to a standardized step current injection protocol with 1000 ms duration. The electrophysiological responses reveal variation in firing rates and spike adaptation, illustrating the diversity in intrinsic excitability across the selected neurons.

The main focus lay on the layer 5 neuron from the primary visual area with experiment ID 488683425, which includes high-quality electrophysiological and morphological data. In addition to this cell, three other neurons—with experiment IDs 485574832, 480353286, and 473601979—were also analyzed to provide comparative context and assess model generalizability across different cell types. These cells represent diverse neuronal subtypes. While detailed transcriptomic characterizations were not used in this project, morphological and electrophysiological variability among these neurons supports their use for comparative modeling.

In the database, electrophysiological data were acquired via whole-cell patch clamp recordings using a range of stimulus protocols such as step currents, slow ramps, and naturalistic noise, which are designed to comprehensively characterize intrinsic neuronal firing properties. A more detailed description can be found in the electrophysiology overview technical whitepaper. Morphological data consist of biocytin-filled neurons imaged and reconstructed in planar images as well as in three dimensions, providing detailed information on cell structure relevant to function. A more detailed description can be found in the morphology overview technical whitepaper.

To illustrate the biological properties of our models, morphological reconstructions and representative electrophysiological traces of the four analyzed neurons are shown above (Figure 1). These visualizations highlight the diversity of cell structures and firing behaviors captured within the dataset.

4 Models

To investigate and replicate the electrophysiological behavior of real neurons, I used both single- and multi-compartment neuron models in Jaxley. These models simulate the membrane potential dynamics of a cell in response to different input currents, using biophysical mechanisms such as membrane capacitance, leak conductance, and voltage-gated ion channels. Both models are based on the Hodgkin–Huxley formalism (Hodgkin and Huxley, 1952), where membrane dynamics are governed by coupled nonlinear differential equations describing the flow of ionic currents across the membrane. While the single-compartment model offers a simplified, computationally efficient representation, the multi-compartment model incorporates detailed morphology to capture spatial aspects of signal propagation. Together, they provide a complementary framework for exploring neural behavior and testing model inference methods.

4.1 Single-Compartment Model

The single-compartment model treats the neuron as a single-point unit, meaning that it assumes a uniform voltage across the entire membrane surface. This simplification allows for efficient simulation of neural dynamics while preserving key biophysical mechanisms. The membrane voltage dynamics are governed by the standard membrane equation:

$$C_m \frac{\mathrm{d}V}{\mathrm{d}t} = -I_{\text{ion}} + I_{\text{ext}}, \tag{1}$$

where C_m is the membrane capacitance, V is the membrane potential, I_{ion} is the sum of ionic currents through membrane channels, and I_{ext} is the externally injected current. In the passive version of the model, only the leak current is included, defined as:

$$I_{\text{Leak}} = g_{\text{Leak}}(V - E_{\text{Leak}}), \tag{2}$$

where g_{Leak} is the leak conductance and E_{Leak} is the leak reversal potential.

For active channels, the ionic currents are generally modeled as voltage-dependent conductances of the form:

$$I_j = \bar{g}_j \, m^M h^N (V - E_j), \tag{3}$$

where $\bar{g}_j$ is the maximal conductance of channel j, m and h are gating variables raised to powers M and N, and E_j is the reversal potential for the ion species. These voltage-dependent currents allow the neuron to generate rich dynamics such as action potentials and subthreshold oscillations, depending on the gating kinetics and conductances.

The cell is built using a custom function that allows flexible inclusion or exclusion of several channel families. The basic passive properties include a membrane capacitance (C_m)—which is fixed to 1.0 throughout this work—, a leak current (parameterized by g_{Leak} and E_{Leak}), an axial resistance—which is fixed to $100\,\Omega \cdot \text{cm}$—, an initial membrane voltage, and a fixed radius of 8 µm. The total length of the compartment is tunable and is a key parameter during calibration. All membrane currents follow the standard Hodgkin–Huxley formalism (Hodgkin and Huxley, 1952), where voltage- and ion-dependent gating variables evolve according to coupled nonlinear differential equations.

In the standard configuration, the model incorporates the following ionic channels and calcium-related mechanisms:

- **Passive:** Leak channel
- **Active:** Fast sodium channel (NaTs2T), delayed rectifier potassium channel (SKv3_1), SK-type calcium-activated potassium channel (SKE2), high- and low-voltage activated calcium channels (CaHVA, CaLVA), calcium pump (CaPump), H-type channel (H), and M-type channel (M)
- **Calcium dynamics:** Nernst-based calcium reversal potential with extracellular and intracellular concentrations ($\text{Ca}_\text{e}^{2+} = 2.0$ mM, $\text{Ca}_\text{i}^{2+} = 5 \times 10^{-5}$ mM)

These channels and mechanisms are adapted from several sources (e.g., Markram et al. (2015); Colbert and Pan (2002); Köhler et al. (1996); Adams et al. (1982); Kole et al. (2006)) and, for simplicity, we will refer to them collectively as Markram-type channels in the following.

In the initial phase of the project, I focused on tuning the parameters of the passive single-compartment model to closely match the voltage response of the corresponding passive multi-compartment model (one that includes morphology but no active channels). Specifically, I aimed to identify values for the model's length and leak conductance g_{Leak} that result in similar voltage behavior, particularly during the rising phase of a step current response.

First, I performed manual tuning on the channel-free single-compartment model to find a suitable length value that allowed it to reproduce the voltage behavior of the multi-compartment model without any channels. Once a good match was obtained, I switched to the full passive setting (only leak channel present), and I tuned both the length and g_{Leak} values with grid search and the sum of squared errors (SSE) loss, comparing the voltage responses given to a standard step current injection (Figure 2). The final parameter values selected in this stage were used throughout the rest of the project as the fixed biophysical baseline of the single-compartment models. I also tried matching the rising voltage phase of an active single-compartment model to an active multi-compartment one, but unfortunately in this case, the other conductance parameters also have an effect on the voltage behavior, so a specific length - g_{Leak} pair is only optimal for a neuron with fixed parameters. As our goal is to find these parameters, we cannot define a pair of length - g_{Leak} values optimal, so we had to stick with the ones found comparing the passive models.

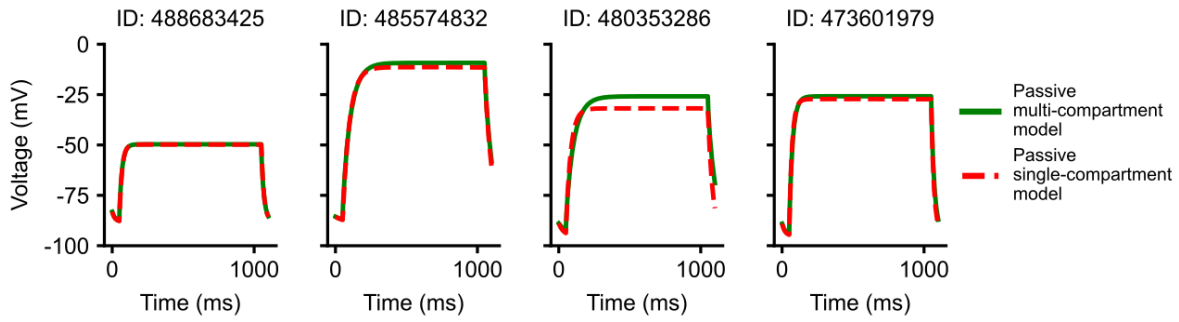

Figure 2: Voltage responses from the passive single-compartment and multi-compartment models after tuning the length and g_{Leak} parameters by grid-search.

Although these tuning steps were empirical in nature, they ensured that the single-compartment model started from a well-calibrated configuration when extended with active channels and the neural ODE framework.

In later stages of the project, the active channel parameters were tuned using gradient-based optimization. The trainable parameters included the conductances of the various ion channels, calcium pump parameters, reversal potentials (e.g., E_{Na}, E_{K}), time constants for calcium channels and the M-type channel, and optionally the compartment length. Each parameter was constrained within biologically plausible bounds. For instance:

- $g_{NaTs2T} \in [0.0, 1.0]$
- $g_{SKv3\_1} \in [0.01, 0.5]$
- $g_{CaHVA} \in [0, 0.001]$
- length $\in [10.0, 400.0]$

The bounds were carefully selected to avoid biologically implausible regimes while allowing enough flexibility for the optimizer to explore diverse solutions.

4.2 Multi-Compartment Model

The multi-compartment neuron model is grounded in the same fundamental biophysical principles as the single-compartment one, but introduces morphological complexity by incorporating detailed neuron morphology reconstructed from experimental data. The model consists of multiple connected compartments representing distinct anatomical regions of the cell, such as the soma, basal and apical dendrites, and axon (Figure 3). This spatial structure enables more accurate simulation of signal propagation, synaptic integration, and dendritic processing.

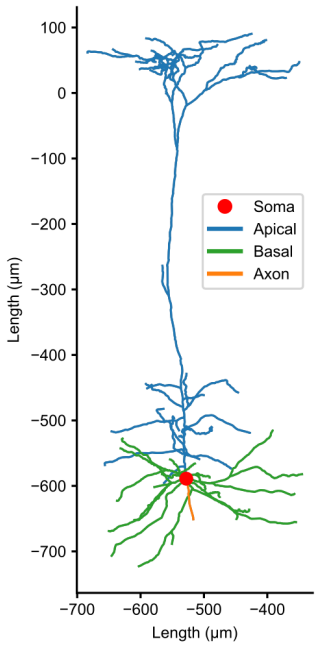

Figure 3: Reconstructed morphology of the 488683425 cell.

As a consequence of the increased complexity, the multi-compartment model includes more parameters, both geometric (e.g., diameters and lengths of individual sections) and biophysical (e.g., conductances of ion channels distributed across compartments). There are 25 free parameters governing the simulations of these models, including ionic reversal potentials, ion channel conductances, calcium dynamics parameters, and time constants for various channel subtypes. These parameters are constrained by biologically informed bounds to ensure physiological plausibility and to stabilize optimization. For example, sodium and potassium reversal potentials are restricted to [-100, -70] mV and [40, 60] mV respectively, reflecting typical intracellular and extracellular ion concentrations. Not all ion channels are present in every section of the neuron; different compartments may express different subsets of channels to reflect their physiological roles. For example, certain potassium and calcium channels are only inserted in the soma and the axon, while dendritic regions may contain specific M-type or H-type channels. Conductance values for each ion channel subtype are set separately for different anatomical compartments, with, for instance, NaTs2T conductance ranging up to 6.0 mS/cm^2 in the soma but only up to 0.04 mS/cm^2 in the apical dendrites, reflecting the known compartmental localization of fast sodium channels.

Some parameters are represented in logarithmic space to account for their wide dynamic range and to improve numerical stability during inference. These include time constants for M-, LVA-, and HVA-type calcium channels, as well as decay constants for calcium pumps. The heterogeneity across compartments adds a layer of realism to the model but also increases the dimensionality of the parameter space, making inference more challenging. Together, these features make the multi-compartment model a powerful tool for simulating realistic neural dynamics and for testing the robustness of parameter inference procedures across different levels of model complexity.

The multi-compartment model used in this work was also implemented in Jaxley, using the same simulator backend as the single-compartment model. However, it uses the morphology files from the Cell Types Database, ensuring a faithful representation of the target neuron's structure. In the first phase of this work, the goal was to match the behavior of the passive multi-compartment model using a simplified single-compartment model. In the second phase, both single- and multi-compartment models were calibrated to reproduce true experimental voltage traces.

5 Passive Model Fitting with Neural ODEs

5.1 Motivation and Setup

In the previous section, I described both a simplified, computationally less expensive single-compartment, and a biophysically detailed multi-compartment neuron model that reflects the true morphology and passive properties of a cortical neuron. While the latter construction offers high biological fidelity, it is rather resource-intensive and contains a large number of free parameters. As a stepping stone toward inferring complex biophysical models from experimental data, I began with a simpler task: fitting a single-compartment model's voltage response to a multi-compartment model's trace. This setup allows us to develop and evaluate our inference techniques under controlled conditions before moving on to considering spiking models and real data.

The central idea in this phase is to approximate the voltage response of the passive multi-compartment model using the simplified passive single-compartment model. Since both models are passive (i.e., they only contain a leak channel but lack voltage-gated ion channels, and so they do not produce any spikes), this task focuses solely on accurately reproducing subthreshold voltage dynamics. The input is a step current stimulus that maintains a fixed amplitude for 1 second—the same protocol that was used in the experimental stimulation of the corresponding real neuron—, and the desired outcome is a close match between the voltage traces of the two models. Due to no action potentials or fast transients being present, the problem is smoother and more tractable than fitting active models, making it an ideal baseline for experimenting with new inference strategies, such as the neural ODE-augmented framework.

5.1.1 The Neural ODE

To bridge the expressiveness gap between the single-compartment and multi-compartment models, we introduce a learnable extension in the form of a neural ODE. The goal is to augment the relatively simple dynamics of the single-compartment model so that it can approximate the behavior of a much more complex and spatially extended multi-compartment neuron. Specifically, we inject a learned external current into the single-compartment model, allowing it to reproduce voltage traces that it would otherwise be incapable of capturing due to its simplified structure.

This added corrective current is generated by a neural network that takes as input a latent variable $u(t)$, and outputs an additive current $I_{\text{add}}(t)$:

$$I_{\text{add}}(t) = \text{NN}_{\text{readout}}(u(t)). \tag{4}$$

We refer to this module as the *readout network*, as it maps internal neural states to added currents.

To make the learned current more flexible and adaptive over time, the latent variable $u(t)$ is defined by a neural ODE, where its dynamics evolve according to a separate neural network:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = \text{NN}_{\text{dynamics}}(V(t), u(t)). \tag{5}$$

We refer to this network as the *dynamics network*, as it determines the temporal evolution of the latent state based on current voltage and latent activity.

The full voltage dynamics of the single-compartment passive model, augmented by this learned current, can be now written as:

$$\frac{\mathrm{d}V}{\mathrm{d}t} = g_{\text{Leak}} \cdot (E_{\text{Leak}} - V(t)) + I_{\text{ext}}(t) + I_{\text{add}}(t), \tag{6}$$

where g_{Leak} and E_{Leak} are the leak conductance and reversal potential, respectively, and $I_{\text{ext}}(t)$ is the externally injected current.

Together, the *readout* and *dynamics networks* form what we call the neural ODE augmentation. This architecture allows the model not only to generate context-dependent corrective currents, but also to internally evolve hidden states over time, potentially capturing temporal dependencies and hidden biophysical processes that are not explicitly modeled.

Importantly, placing $u(t)$ within a differential equation, rather than treating it as a free latent variable at each time step, enforces temporal consistency and smoothness. It also aligns with the goal of modeling biological processes, many of which are continuous and governed by differential equations. This setup enables the use of neural ODE techniques and allows the augmented model to generalize more robustly,

particularly when scaling to longer time windows or more diverse input stimuli. Additionally, by embedding $u(t)$ in a differential framework, we avoid unrealistic discontinuities in the added corrective signal and improve numerical stability during training.

This approach serves as a modular and extensible foundation for later phases of the project, where more complex biophysical behaviors (including spiking) and structural parameters will be incorporated and optimized jointly with the neural ODEs.

5.1.2 Variants of the Latent Dynamics

Our initial experiments consider two forms of the latent variable u. In the simpler case, u is defined directly as a sequence of free parameters, one per time step, effectively decoupling the latent state evolution from any underlying dynamics. Optionally the current time t is also included as an additional input at every time step. These setups serve as benchmarks to evaluate how much improvement can be gained purely from using this uncoupled latent state evolution. In the second, more biologically motivated formulation, u is governed by a differential equation, and the readout network outputs the added current as a function of $u(t)$. This allows us to explore more realistic scenarios in which the latent state evolves smoothly and continuously over time, as would be expected in a physical system.

This fitting task also serves as a controlled environment where we can explore several auxiliary questions that will be relevant throughout the project. For example, we investigate whether incorporating time explicitly into the neural ODE system improves performance, how the dimensionality of the latent variable u affects the model's capacity and stability, and whether certain hyperparameter settings (such as layer width or learning rate) result in more robust convergence. Finally, this stage allowed us to identify and address an important instability issue: the divergence or explosion of $u(t)$ values. This led us to incorporate initialization strategies inspired by recent work on stable neural ODEs.

In summary, this phase of the project is not only about reproducing passive voltage dynamics as perfectly as possible, but also about developing, testing, and refining the architecture of our neural ODE-augmented model and inference pipeline. The methods developed here will be carried over to more complex settings involving multi-compartment, spiking neurons and real experimental data, where similar challenges—such as hidden dynamics, parameter degeneracy, and nonlinearity—will be even more pronounced.

5.2 Added Current without Differential Evolution

In this section, I describe the initial approach to learning the voltage response using a latent variable u that is updated at each time step by a neural network. While u may depend on inputs such as time, voltage, or its own previous value, its evolution is not governed by an explicit differential equation or continuous-time dynamic. Instead, its update is handled entirely through parametric transformations applied independently or recurrently at each time step.

This formulation allows the model to learn time-dependent or voltage-conditioned additive currents without requiring integration of dynamical equations. As such, this setup serves as a practical intermediate between purely static input mappings and more complex dynamical latent models explored later in the project.

5.2.1 Model Architecture and Training Setup

The augmented mechanism is implemented as a modular system composed of two feedforward neural networks—as described in Section 5.1.1: a latent dynamics network and a readout network. At each time step t, the latent network receives a set of inputs and returns an updated latent vector $u(t)$. This updated latent state is then passed through the readout network, which produces an additive current term $I_{\text{add}}(t)$. This current is injected into a single-compartment passive neuron model alongside the predefined step stimulus.

In the majority of my experiments, the latent dynamics network contains two hidden layers with 64 and 32 units respectively, each followed by ReLU activations, and outputs an 8-dimensional latent vector. The readout network maps this latent vector to a scalar output using a hidden layer of size 32 and a ReLU activation. To ensure the added current remains within a biologically plausible range, it is scaled by a fixed factor during training. This scaling (which can be interpreted as the standard deviation of

the added current, and during my experiments is usually set to values between 0.01 and 0.1) prevents the model from introducing overly strong or destabilizing input currents during simulation. In most experiments, the voltage trace $v(t)$ is also normalized to the range $[-1, 1]$ before being passed to the network.

The model is trained to minimize the sum of squared errors (SSE) between the predicted voltage trace of the augmented passive single-compartment model and the reference voltage trace (from the passive multi-compartment model). Optimization is performed using the Adam optimizer (Kingma and Ba, 2017). I experimented with both constant and exponentially decaying learning rates, as well as L-BFGS (Nocedal, 1980), though the latter proved to be substantially slower and less effective in my trainings.

5.2.2 Input Variants and Results

I tested three configurations regarding which inputs are provided to the latent dynamics network:

- **Time-only input:** Using only the normalized time t as input, the model achieved a minimum loss of 130 after 200 epochs (learning rate: 1.2×10^{-2}). Extending training to 500 epochs further reduced the loss to 11.
- **Voltage, previous latent state, and time as input:** When all three signals were provided to the network input, we observed the best performance. With an initial learning rate of 1.5×10^{-2} and a decay factor of 0.99 per epoch, the model reached a minimum loss of 7.5 after 200 epochs.
- **Voltage and previous latent state only (excluding time):** As expected, not using the time variable as input drastically degraded performance. Even after prolonged training with a learning rate of 1×10^{-4}, the lowest achievable loss remained at 446.

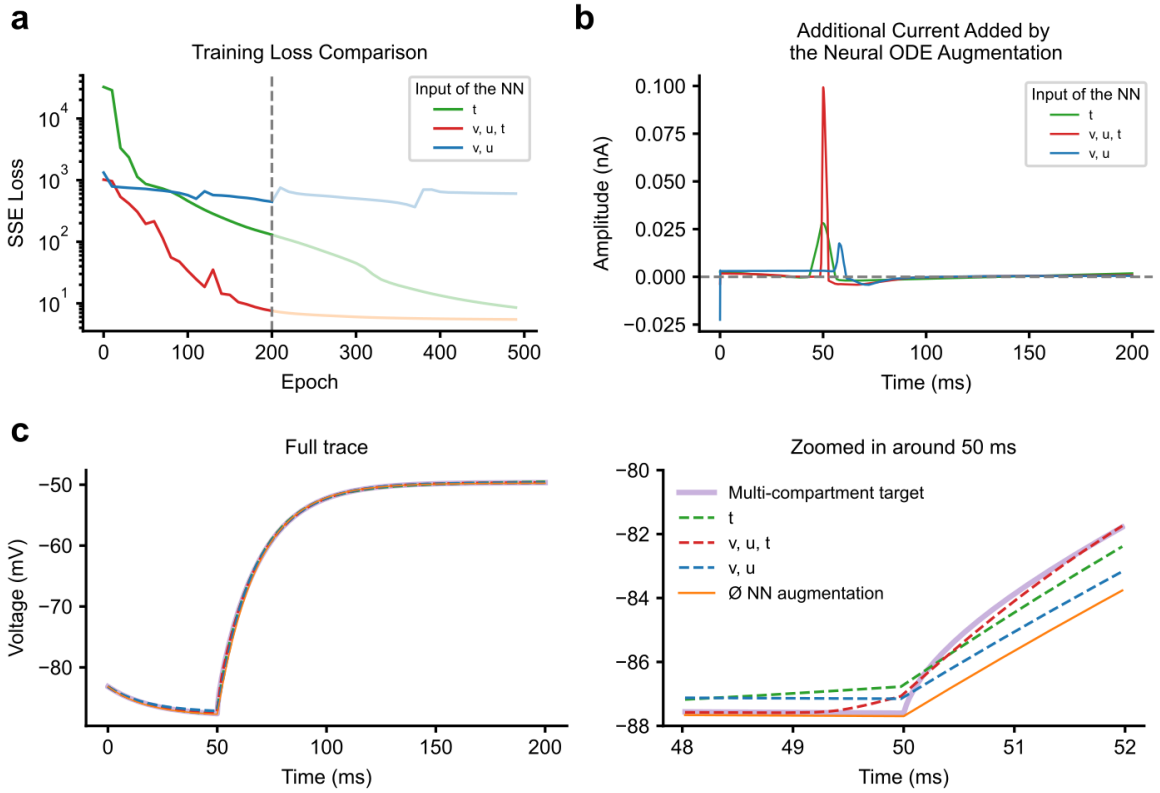

Figure 4: Results after training without differential evolution in the latent state with different inputs for the latent dynamics network. **Panel a:** Training loss with the three input variants after 200 and 500 epochs. **Panel b:** The additional current added by the neural ODE augmentation after training for 500 epochs with the 3 input variants. **Panel c and d:** Voltage response from the passive multi-compartment model and the passive single-compartment models with and without the trained added current.

5.2.3 Interpretation

These results suggest that including explicit time information in this setup enables the network to learn additive currents that follow the temporal patterns of the input stimulus and resulting voltage. Conversely, excluding time prevents the network from identifying when a modification in the external stimulus is needed, leading to suboptimal performance even with extended training. Interestingly, while the latent vector u provides additional capacity, it alone cannot replace the temporal signal.

These experiments served as an important testbed for refining the model architecture and understanding the role of time and voltage as conditioning signals in discrete-time latent state evolution. They showed that the latent states can already capture stimulus- and voltage-dependent adjustments without requiring explicit differential dynamics. Importantly, they reveal that some form of time input is essential for learning structured additive currents, while voltage alone is insufficient. While this discrete-time formulation performed well in practice, it does not reflect the continuous-time nature of neural processes. For this reason, the next section moves toward a biologically motivated formulation where the latent state evolves according to a differential equation, although this introduces additional complexity and does not necessarily improve training performance.

5.3 Added Current with Differential Evolution

In the previous section, the latent state u was updated directly through parametric transformations without being governed by explicit dynamics. While this formulation allowed flexible additive currents, it lacked the temporal continuity that characterizes biological processes. From a biological perspective, neural states evolve continuously in time, shaped by differential equations describing ion channel kinetics, membrane currents, or other dynamical processes. In order to better align with this principle, in this section I define the latent state u through a differential equation whose evolution is parameterized by a neural network. Although this shift increases the biological plausibility of our model, it also introduces new challenges, particularly regarding stability.

5.3.1 Motivation

Introducing a differential equation for the latent state u brings the model closer to biological realism by enforcing continuous-time evolution. In real neurons, hidden processes such as synaptic conductances, ionic currents, or dendritic voltage propagation do not jump arbitrarily from one value to another at discrete timesteps, but instead change smoothly according to underlying biophysical mechanisms. Modeling u with a differential equation ensures that the latent dynamics evolve smoothly over time and are constrained by continuous-time laws, making them physiologically interpretable.

The motivation for this formulation is therefore not to achieve more efficient training or faster convergence compared to the stepwise update rule, but to incorporate the augmented model within a structure that reflects how biological systems operate in the real world. As we will see in this section, introducing ODE-governed latent states often complicates optimization, as small instabilities in the learned dynamics can accumulate and lead to exploding trajectories. However, this difficulty highlights the central trade-off of biologically grounded modeling: the closer one aligns with physiology, the more important—but also the harder—it becomes to address stability and robustness in the underlying dynamical system.

5.3.2 Model Architecture and Training Setup

The augmented mechanism again consists of a latent dynamics network and a readout network. However, in this case, the neural network defines the right-hand side of a differential equation:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = f_\theta(u, v),$$

where f_θ is parameterized by the latent dynamics network and depends on both the current latent state u and the membrane voltage v (in one experiment I also tried adding the time step t as input). The resulting latent trajectory $u(t)$ is then numerically integrated jointly with the single-compartment neuron model. In other words, the evolution of u unfolds in parallel with the membrane potential dynamics, ensuring that both processes remain coupled over time. At each time step, the readout network maps

$u(t)$ to an additive current $I_{\text{add}}(t)$, which is injected into the neuron model in the same way as in the discrete, non-differential case.

Apart from this change in how u evolves, the overall architecture is the same as in the previous setup. We continue to use an 8-dimensional latent state, a multilayer latent dynamics network, and the same readout structure. The training objective also remains unchanged: we aim to fit the voltage trace of a passive single-compartment neuron to that of the corresponding passive multi-compartment model. Thus, the only substantive modification is that the latent state evolution is now governed by continuous-time dynamics rather than stepwise updates, allowing us to see how this added biological realism affects both performance and stability.

5.3.3 Experimental Variants and Results

I explored several variations of this differential latent setup to understand how the added dynamics shape the augmented learning process. One natural question in defining the latent dynamics is whether the neural network should receive the explicit simulation time t as an input, alongside the current latent state $u(t)$ and the voltage $v(t)$. Intuitively, conditioning on t might help the network represent non-stationary dynamics, for example added currents that evolve differently across distinct phases of the simulation. On the other hand, excluding t forces the system to rely purely on the evolving latent state $u(t)$, which can be seen as a more autonomous and biologically inspired formulation, that also prevents overfitting.

In practice, the results were somewhat unexpected. Without including t, the model reached a loss of 37 after 500 epochs and as low as 17 after 1000 epochs (Figure 5). When time was included as an input, the corresponding values were a little higher (around 65 after 500 epochs and 30 after 1000 epochs). These outcomes suggest that, at least under the current setup, the explicit time input did not provide a clear advantage.

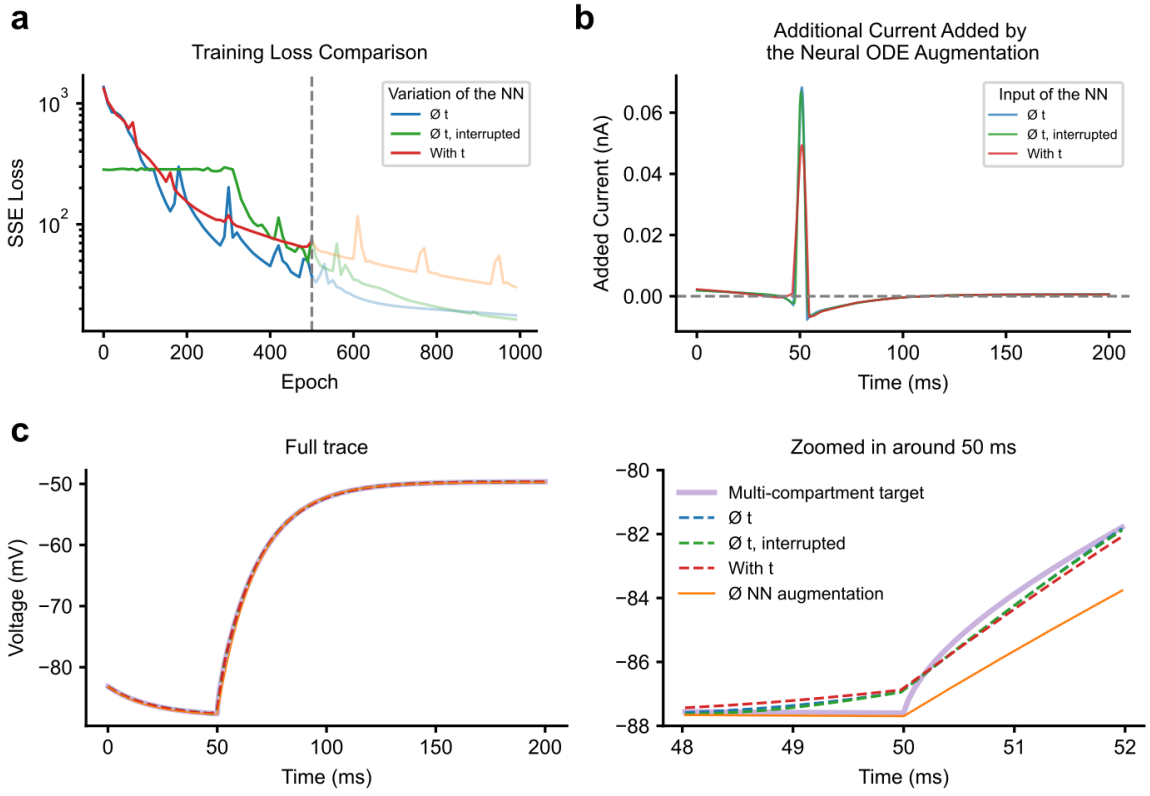

Figure 5: Results after training with differential evolution in the latent state with different inputs for the latent dynamics network. **Panel a:** Training loss with the three setup variants after 500 and 1000 epochs. **Panel b:** The additional current added by the neural ODE augmentation after training for 1000 epochs with the three setup variants. **Panel c and d:** Voltage response from the passive multi-compartment model and the passive single-compartment models with and without the trained added current.

There are several possible reasons for this behavior. One factor may be the choice of learning rate and the normalization of t, which strongly affect the stability of training. Another consideration is that, by providing absolute time, the network may have relied too heavily on this direct signal rather than developing richer latent dynamics, leading to slower optimization. Conversely, when trained without t, the network had to encode all relevant temporal structure within the latent state itself, which may have promoted more efficient representations. Although these explanations remain somewhat speculative, the contrast highlights the sensitivity of neural ODE formulations to design choices that at first appear minor.

I also experimented with an interrupted training strategy, in which the simulations during training were stopped early if the loss rose above a predefined threshold. This approach naturally reduced computational cost in the early epochs and sped up training. However, the final performance was very similar to the uninterrupted case (loss of 16 after 1000 epochs), and thus I did not pursue this strategy in later experiments.

Beyond the time-conditioning experiments, I also examined how the size of the latent variable u and the width of the dynamics network affect the training performance. Specifically, I compared latent vectors with dimensions both smaller and larger than the baseline of 8, and varied the hidden layer width around the default choice of 32. The results were consistent: reducing either the latent dimension or the layer width led to noticeably worse performance, while increasing them did not provide significant improvements in terms of final loss or the structure of the added currents. Based on these observations, I kept the default configuration ($\dim(u) = 8$, layer width $= 32$) for subsequent experiments as a balanced choice.

Finally, as a step towards fitting active models, I attempted to extend the training procedure by optimizing not only the neural network parameters, but also the intrinsic passive properties of the single-compartment model, in this case the length and the leak conductance g_{Leak}. As expected, jointly training these continuous parameters with the latent dynamics substantially increased the difficulty of the optimization. Nevertheless, the network was able to converge to a solution with a loss around 290, which, although higher than in the fixed-parameter case, represents an encouraging step toward future experiments where the goal will be to infer not only additive currents but also the biophysical channel parameters of active models.

5.3.4 Instability in the Latent Dynamics: A Stable Initialization

While the introduction of a differential latent variable $u(t)$ allows the augmented network to capture temporal dependencies in a more structured and biologically grounded way, it also introduces a nontrivial stability issue. Already before any training, simulations revealed that the components of the latent vector u tend to diverge rapidly over time. Figure 6 illustrates this phenomenon: for an 8-dimensional latent state, each component grows exponentially during the course of the simulation. The rate of divergence depends strongly on the hyperparameter $u_{\text{time\_constant}}$, which scales the update rule:

$$u_{\text{new}} = u + \frac{\Delta t}{u_{\text{time\_constant}}} \, \text{NN}_{\text{dynamics}}(v(t), u(t)),$$

where Δt denotes the simulation time step, and $u_{\text{time\_constant}}$ is a scaling parameter that controls how quickly the neural network output influences the latent dynamics. As shown in the figure, smaller values of $u_{\text{time\_constant}}$ accelerate the divergence, while larger values only delay but do not fully prevent it. This exponential instability posed a fundamental obstacle for both interpreting and training the model.

My initial attempts to mitigate the issue highlighted the severity of the problem. Replacing the ReLU nonlinearity with a bounded activation such as the sigmoid reduced the growth rate, leading to trajectories that diverged linearly instead of exponentially. However, the latent state still failed to remain bounded, and the resulting added currents became unrealistic over longer simulation periods. These observations indicated that the instability was not simply a matter of nonlinearity choice, but instead related to the underlying numerical properties of the update rule itself.

Faced with this instability, I turned to the literature to investigate whether similar issues had been encountered in related contexts. In particular, I found the work of Westny et al. (2024), which specifically addresses the interplay between numerical integration techniques, stability regions, step size, and initialization strategies in the training of neural ODEs. Their analysis demonstrates that the choice of solver and its associated stability region implicitly constrain the range of parameter values that can be learned effectively, as the eigenvalues of the linearized dynamics must remain compatible with the numerical

integration method. Based on these insights, they propose a stability-informed initialization procedure that aligns the spectrum of the system's Jacobian with the stability region of the chosen integrator, thereby ensuring numerical stability while improving both training efficiency and prediction accuracy. Motivated by these results, I adopted this initialization strategy in my own setting and evaluated how it impacts the stability of the latent dynamics. In the following, I describe the details of this method and explain how it was applied in my experiments.

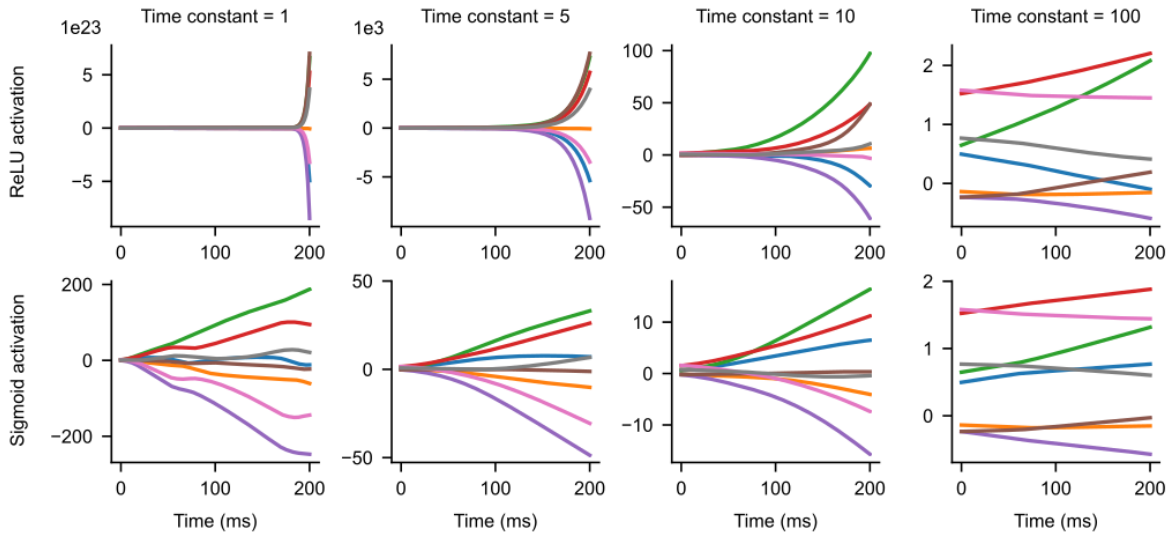

Figure 6: Untrained latent trajectories over 8002 simulation steps (200 ms) for different values of $u_{\text{time\_constant}}$ using ReLU and sigmoid activations. With ReLU, the latent components diverge exponentially, and smaller $u_{\text{time\_constant}}$ values amplify the instability. Using a sigmoid activation function reduces the growth to linear divergence, but the latent state still does not remain bounded.

A Stability-Informed Initialization based on Westny et al. (2024)

To understand the idea behind the stability-informed initialization, it is useful to recall what stability means for both the dynamical systems themselves and for the numerical solvers that approximate them. A continuous-time system can generally be written as:

$$\dot{x} = f(x, u; \theta), \tag{7}$$

with x denoting the state, u an external input, and θ the parameters. Stability in dynamic systems means that the system either returns to equilibrium or maintains its state of equilibrium after perturbations (Ljung, 1999). For linear systems of the form:

$$\dot{x} = Ax + Bu \tag{8}$$

this reduces to a simple eigenvalue condition: the system is exponentially stable if all eigenvalues of A lie in the left half of the complex plane.

In practice, however, such systems are rarely solved analytically and must be integrated numerically. This brings in a second layer of stability: that of the numerical method itself. Explicit solvers such as Runge–Kutta methods—for which the 1st order RK is known as Euler Forward (EF) and is used throughout my thesis—, are only stable when the products of step size h and eigenvalues λ_i lie within a so-called stability region. These regions can be visualized in the complex plane and expand as one moves from the simple Euler Forward method to higher-order Runge–Kutta methods (panel a of Figure 7). The stability condition can be written as:

$$\left| 1 + h\lambda_i + \frac{(h\lambda_i)^2}{2!} + \cdots + \frac{(h\lambda_i)^p}{p!} \right| \leq 1 \tag{9}$$

which shows how the choice of solver and step size interacts with the system's eigenvalues. Intuitively, even if the underlying dynamics are stable, the numerical solver may drive trajectories to divergence if

$h\lambda_i$ falls outside its stability region. This interplay between system eigenvalues, solver, and step size forms the basis for stability-informed initialization.

Implementation: In the context of our work, the function f in the dynamical system can be represented by a feedforward neural network of depth n:

$$f(x) = W_n\,\sigma\big(\cdots\sigma(W_1 x + b_1)\cdots\big) + b_n, \tag{10}$$

where σ is a nonlinear activation function, $W_i \in \mathbb{R}^{d_{h_i}\times d_{h_{i-1}}}$ and $b_i \in \mathbb{R}^{d_{h_i}}$ are the weight matrices and bias vectors, and d_{h_i} is the hidden dimension of the i-th layer. The input to the network is given by $x = \mathtt{x} \oplus u$, i.e., the concatenation of the system state $\mathtt{x}$ and the external input u.

To analyze the stability of the network more formally, we consider the linearization of the neural dynamics around a reference state. Assuming the biases b_i are small and the activation function σ has a linear region with a slope of κ (which is the case for ReLU and its variants), the Jacobian of the system with respect to the state can be approximated as:

$$\frac{\partial f(x)}{\partial x} \approx \kappa^n W_n W_{n-1}\cdots W_1 = A, \tag{11}$$

which corresponds to an n-layer linear feedforward network. For activation functions with non-unit slopes at the origin, the matrices W_i can be rescaled by the local slope to ensure the linear approximation remains valid. This way, we assume $\kappa = 1$ from now on without loss of generality.

This linearized system provides a first-order estimate of the eigenvalues that will govern the local stability of the dynamics. We have to keep in mind that stability of the linearized system does not strictly imply stability of the full nonlinear network, but it serves as a useful guideline for initialization: eigenvalues of A lying outside the numerical stability region of the chosen solver will likely produce diverging trajectories.

If the weights and biases are initialized in an uninformed, random manner, the (linear approximation of the) resulting network will with high probability violate the stability constraints described in the previous section, which has direct consequences. To verify whether this instability arises in practice, I examined the problem in my own setup. I found that with random initialization of the dynamics neural network, the scaled eigenvalues of the system were not all located inside the stability region of the Euler Forward method (panel b of Figure 7). This led to exploding trajectories even before any training (Figure 6). To check whether this behavior is more general, I implemented a simple minimal neural ODE in PyTorch consisting of a two-dimensional input, a single hidden layer, and a ReLU activation function.

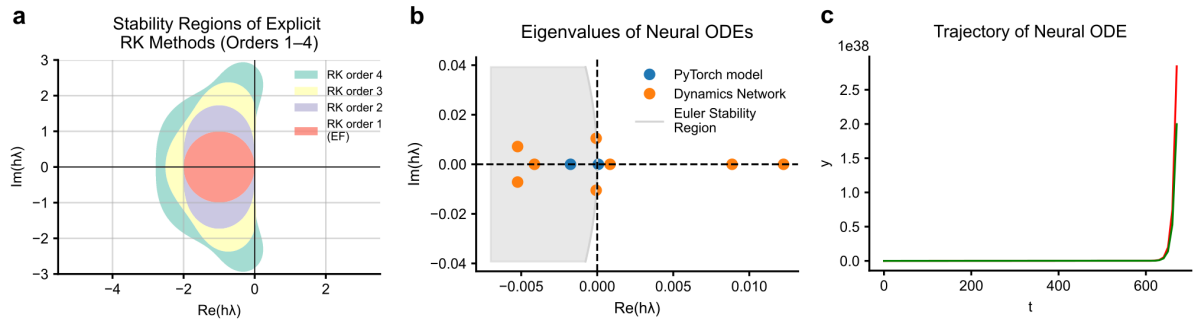

Figure 7: **Panel a:** Stability regions of explicit Runge-Kutta methods of order 1 to 4, computed using the standard stability condition (9). **Panel b:** Eigenvalues of the linearized system for the randomly initialized toy model and the latent dynamics network. **Panel c:** Trajectory showing exponential divergence of the toy model neural ODE.

Formally, the minimal system can be written as:

$$\dot{y}(t) = f_\theta(y(t)) = \mathrm{NN}(y(t)), \tag{12}$$

where $y(t) \in \mathbb{R}^2$ is the state vector and NN denotes the feedforward neural network parameterized by θ. Here, the network outputs the time derivative of the state, and integration of this system with the Euler Forward (EF) method produces the trajectories shown in panel c of Figure 7. Despite the simplicity of this setup, the eigenvalues of the linearized system already reveal violations of the EF stability region,

demonstrating that naive random initialization can lead to divergent dynamics even in small-scale neural ODEs. These experiments indicate that the instability is not specific to my model, but rather a general problem that needs to be addressed.

To mitigate the divergence issue observed in the previous experiments, the authors employ a stability-informed initialization for the weights and biases of the neural network. The key idea is to construct weight matrices such that the linearized system has eigenvalues inside the stability region of the numerical solver.

For a simple case where the desired eigenvalues of the linearized system A are real, let $W_i \in \mathbb{R}^{d_x \times d_x}$ be identical diagonal matrices:

$$W_1 = \cdots = W_n = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_{d_x})^{1/n}. \tag{13}$$

This ensures that the linearized system $A = W_n W_{n-1} \cdots W_1$ has the earlier selected eigenvalues λ_j. For complex eigenvalues $\lambda_k \in \mathbb{C}$, and the n-th principal root being $\lambda_k^{1/n} = \mu_k + \omega_k i$, real matrices W_i are constructed in block-diagonal form using 2×2 blocks:

$$J_{\lambda_k} = \begin{bmatrix} \mu_k & \omega_k \\ -\omega_k & \mu_k \end{bmatrix}, \tag{14}$$

so that each block has eigenvalues $\mu_k \pm \omega_k i$. Each weight matrix W_i is then defined to be the direct sum of these blocks:

$$W_1 = \cdots = W_i = J_{\lambda_1} \oplus J_{\lambda_2} \oplus \cdots \oplus J_{\lambda_{d_x/2}}, \tag{15}$$

which guarantees that the compound matrix A has the desired eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_{d_x}$.

To handle layers with dimensions different from d_x, to avoid sparse weight matrices, and to add randomness to the initialization strategy, the authors generalize this procedure using:

$$A = \underbrace{\Lambda_n \Pi_{n-1}^{-1}}_{W_n} \underbrace{\Pi_{n-1} \Lambda_{n-1} \Pi_{n-2}^{-1}}_{W_{n-1}} \cdots \underbrace{\Pi_2 \Lambda_2 \Pi_1^{-1}}_{W_2} \underbrace{\Pi_1 \Lambda_1}_{W_1}, \tag{16}$$

where Λ_i incorporates the desired eigenvalues and dimension adjustments, and Π_i are random orthogonal matrices sampled from the Haar distribution. Importantly, the inclusion of Π_i does not alter the eigenvalues of the overall system, but ensures that all entries $w_{j,k}$ in W_i are nonzero with probability 1.

To include external inputs, the vector u is concatenated with the state vector $\mathbf{x}$ prior to propagation. This is accounted for by extending Λ_1 such that $\Lambda_1 \in \mathbb{R}^{d_{h_1} \times (d_x + d_u)}$. Considering for example the simple case when $d_x = 2$ and $d_{h1} = 4$, and $d_u = 1$, the extended Λ_1 matrix would look like the following:

$$\Lambda_1 = \begin{bmatrix} \mu_1 & \omega_1 & 0 & 0 \\ -\omega_1 & \mu_1 & 0 & 0 \\ \nu_1 & \nu_2 & 0 & 0 \end{bmatrix}^T, \tag{17}$$

where $\nu_k \sim \mathcal{U}(-\zeta, \zeta)$ and $\zeta = \frac{1}{n} \sum_{i=1}^{n} |\mu_i|$.

Although the linearization is valid for $b_i \approx 0$, initializing $b_i = 0$ can hinder learning. Instead, biases are sampled from a small uniform range:

$$b_i \sim \mathcal{U}(-\epsilon \mathbf{1}_{d_{h_i}}, \epsilon \mathbf{1}_{d_{h_i}}), \tag{18}$$

where $\mathbf{1}_{d_{h_i}}$ is a vector of ones of size d_{h_i} and ϵ is tunable, I used $\epsilon = 10^{-4}$ similarly to the authors.

The final step in the stability-informed initialization is to specify the eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, for which then we compose the necessary weight matrices. Thus the initialization procedure is completed using a simple rejection sampling algorithm. First, real and/or complex-conjugate eigenvalue pairs are sampled within the stability region of the chosen solver for a given step size, based on the stability condition (9). These eigenvalues are then used to construct the matrices Λ_i as described above. Random orthogonal

matrices Π_i are sampled and combined with Λ_i to form the weight matrices W_i as in (16). Finally, the bias terms are initialized according to (18).

Overall, this stability-informed initialization procedure provides a principled way to initialize the weights and biases such that the linearized dynamics respect the stability constraints imposed by the Euler Forward solver, thereby reducing the risk of unstable latent state trajectories and improving convergence during optimization in practice. Figure 8 shows the effect of this initialization on the latent dynamics for different values of $u_{\text{time\_constant}}$ using my neural ODE-augmented model, both before and after training.

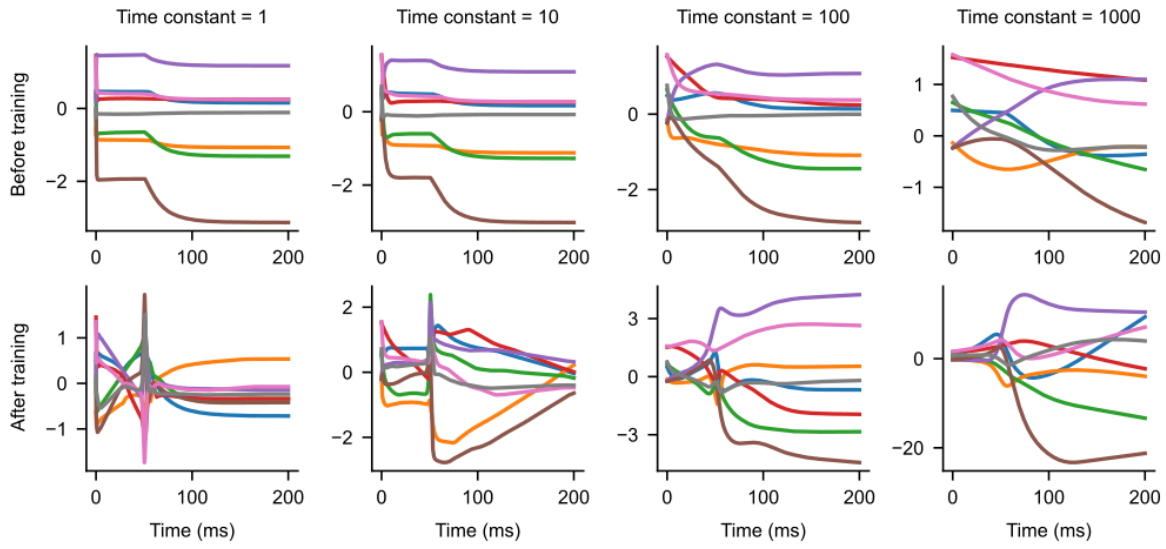

Figure 8: Latent trajectories over 8002 simulation steps (200 ms) for different values of $u_{\text{time\_constant}}$ using the stability-informed initialization technique, before and after training the model. All trajectories remain stable, and larger values of $u_{\text{time\_constant}}$ lead to slower changes in the latent vector.

5.3.5 Interpretation

Introducing differential evolution for the latent variable u provides a more biologically motivated formulation by embedding its dynamics into continuous time. In this setup, the latent state evolves according to a differential equation rather than discrete stepwise updates, allowing the model to capture temporal dependencies in a smoother and more biologically realistic manner. This not only enforces continuity in the latent trajectory, but also enables the neural ODE augmentation to interact with the voltage trace on different timescales, controlled by the $u_{\text{time\_constant}}$ parameter of the latent dynamics.

At the same time, the added complexity introduces new challenges. As shown, training becomes more sensitive to initialization and hyperparameters, particularly those that govern stability of the latent dynamics. While the continuous-time formulation can, in principle, provide more interpretable additive currents, in practice,—due to its complexity—, it did not outperform the simpler discrete-time setup. Instead, its main advantage lies in aligning the latent state evolution with biological intuition, making it a valuable framework for extensions where stability, long-term temporal structure, or even interaction with additional dynamical processes are essential.

Overall, these results highlight a trade-off: discrete-time updates of the latent state offer simplicity and strong empirical performance, whereas differential dynamics add interpretability and biological plausibility at the cost of training complexity. The stability-informed initialization procedure introduced here mitigates many of the instability issues that would otherwise arise, making the continuous-time approach both more practical and more robust during optimization. This opens the possibility that, in future experiments, the differential equation–governed augmentation, when combined with this initialization, can produce stable and accurate results.

6 Active Model Fitting to Experimental Data

6.1 Motivation and Setup

Having established the neural ODE framework on passive neuron models, I next applied it to active single- and multi-compartment models in order to fit experimental recordings (x_o). Unlike the passive case, active models incorporate voltage-gated ion channels, enabling them to generate action potentials. This introduces both new opportunities and challenges: while spikes are essential for capturing the rich dynamics observed in real neurons, they also represent abrupt, high-amplitude changes in the voltage trace, which are notoriously difficult to fit.

First, I constructed an active single-compartment model that extends the leak-only configuration by including sodium, potassium, calcium, and additional channels such as H- and M-type channels. In particular, I initially focused on one experimental recording from the neuron with ID 488683425. Since the first $37,5\,\text{ms}$ of the recording contains no spikes or relevant activity, I restricted the analysis to the following $42,5\,\text{ms}$ which includes a single spike. For consistency, I applied this same cut-off in all subsequent experiments, as no relevant events occur during this initial window in any of the cells. However, for simplicity of presentation, I always refer to the total length including the cut-off—for instance, in this case I denote the recording as $80\,\text{ms}$. Fitting such data requires precise alignment between simulated and experimental spike timings, as even small temporal shifts can lead to large discrepancies under standard loss functions (e.g., the SSE used earlier).

To address this issue, I used the *soft dynamic time warping* (soft-DTW) loss, which allows for flexible alignment between traces while remaining differentiable and thus compatible with gradient-based training. This enables the fitting procedure to focus on the similarity of the spike waveform and overall dynamics, rather than being overly penalized by small temporal misalignments.

In practice, I trained $n_{\text{particles}} = 300$ models in parallel, each initialized with different parameter settings, and evaluated their performance under the soft-DTW loss. This parallelized optimization was necessary due to the sensitivity of active model fitting and the highly non-convex nature of the loss landscape. As will be shown, the spike remains the most challenging feature to reproduce, motivating several modifications to both the model architecture and the optimization procedure.

In addition to the single-compartment case, I also tested the framework on the full multi-compartment cell model to assess whether the added biophysical detail could improve the fit to x_o. While this increased expressivity allowed the model to capture certain features more naturally, it also introduced computational challenges, without yielding substantially better results compared to the simpler single-compartment formulation.

To further enhance the expressivity of the single-compartment model, I later experimented with alternative channel formulations inspired by the Pospischil-type reduction described in Pospischil et al. (2008). These simplified but biologically motivated sodium, potassium, and calcium channels provided a more compact parameterization and allowed exploration of different gating dynamics. With these models, I investigated modifications of the channel gating functions by adding interpolated corrections to the underlying gating curves, and trained the model to learn the interpolation points directly. Together, these approaches broadened the space of candidate models and highlighted the trade-off between model complexity, biophysical plausibility, and trainability in active model fitting.

6.1.1 Soft-DTW Loss

Dynamic time warping (DTW) is a widely used algorithm for measuring the similarity between sequences of points, typically representing time series (Sakoe and Chiba, 1978). In general, DTW identifies the best alignment between two sequences, permitting flexible correspondences that handle different lengths and temporal shifts. Formally, given two sequences $x = (x_1, \ldots, x_T)$ and $y = (y_1, \ldots, y_{T'})$, the dynamic time warping cost is defined as:

$$\text{DTW}(x, y) = \min_{\pi \in \mathcal{P}} \left(\sum_{(i,j)\in\pi} \big(c(x_i, y_j)\big)^p \right)^{\frac{1}{p}}, \tag{19}$$

where $\pi = \big((i_1, j_1), \ldots, (i_L, j_L)\big)$ is an alignment path of length L, $\mathcal{P}$ is the set of all admissible paths, $c(\cdot, \cdot)$ is a local cost function, and p controls the ℓ_p norm used to aggregate local costs along the path. During my work I fixed $p = 1$, which will be used in the following. A path is admissible if it:

(i) starts by matching the first elements and ends by matching the last ones,

(ii) is monotonically increasing in both indices,

(iii) includes all indices of both sequences at least once.

This optimization problem can be solved using a simple dynamic programming recurrence in $\mathcal{O}(TT')$ time. In practice, faster approximate methods also exist, and many programming libraries provide efficient implementations of DTW for practical use.

While DTW provides flexibility in aligning sequences, it is inherently non-differentiable due to the min operator, making it unsuitable for gradient-based training. The soft-DTW formulation overcomes this limitation by replacing the hard minimum in the recurrence with a smooth soft-min operator, yielding a differentiable approximation that retains the alignment flexibility of DTW while enabling backpropagation (Cuturi and Blondel, 2018).

Formally, soft-DTW can be computed as:

$$\text{soft-DTW}_\gamma(x, y) = -\gamma \log \sum_{\pi \in \mathcal{P}} \exp\left(-\frac{1}{\gamma} \sum_{(i,j)\in\pi} c(x_i, y_j) \right), \tag{20}$$

where $\gamma > 0$ controls the smoothness of the approximation, interpolating between classical DTW as $\gamma \to 0$ and an average over all alignment paths as $\gamma \to \infty$. In practice, we apply windowed reduction and additional path penalties to focus on spike regions and stabilize training.

This differentiable alignment is particularly useful for fitting spiking neuron models. It enables the loss to focus on the similarity of spike waveforms and overall voltage dynamics, while tolerating small temporal shifts in spike timing across simulations. By emphasizing meaningful dynamical differences rather than exact time-point matches, soft-DTW provides a robust and flexible objective for gradient-based training.

6.2 Single-Compartment Model Fitting

As described above, in this part I focused on fitting a single-compartment model to a 80 ms experimental voltage recording from ID 488683425 containing a single spike. While soft-DTW provides a differentiable loss that allows flexible alignment between simulated traces and x_o, accurately reproducing the spike remains challenging due to its rapid voltage changes compared to non-spiking regions. To explore the space of possible solutions and account for the highly non-convex loss landscape, I trained $n_{\text{particles}} = 300$ models in parallel with different, randomly initialized parameters.

The following experiments address key aspects of this fitting challenge. First, I investigated modifications to the training procedure, including adaptive time steps and using two separate neural networks to handle the distinct dynamics of spiking versus non-spiking regions. Next, I computed the optimal added current to gain insight into how the model would need to adjust channel contributions to minimize the loss. Finally, I extended the training and analysis to longer simulation periods with multiple spikes, examining both the limitations of the single-compartment model and the behavior of initial parameterizations. These experiments highlight the interplay between model expressivity, training strategy, and the inherent difficulty of reproducing action potentials, setting the stage for subsequent multi-compartment fitting.

6.2.1 Handling Spiking Regions: Adaptive Time Steps and Two Separate Neural Networks

Reproducing the spike in the single-compartment model proved particularly challenging due to the rapid voltage changes during the action potential, which are much larger than in the non-spiking regions. To address this, I explored two complementary strategies aimed at improving the model's ability to fit both spiking and non-spiking dynamics. In all of these experiments, I completed the loss by adding a small regularization term that encouraged the models to produce a spike: the absolute difference of the maximum values in the simulated trace and x_o

The first approach involved the use of an adaptive simulation time step: the baseline $dt = 0.025$ ms was reduced to $dt = 0.0125$ ms when a spike was detected (i.e., the voltage crossed a specific threshold) and the higher value was used in the remaining regions. This aimed to provide finer temporal resolution where the voltage changes were steep, improving gradient accuracy in the critical regions. While this

method did slightly improve spike alignment, it significantly increased computation time and was difficult to implement reliably.

The second approach used a modified neural network architecture, in which I created two-two identical dynamics and readout networks: one pair of networks is active during spikes and the other during non-spiking periods. This allows the model to specialize each network to the dynamics of its respective regime. Compared to the adaptive *dt* approach, this method was easier to implement, trained in a reasonable time, and achieved better spike fits. Consequently, this 2-network framework was adopted for all subsequent single- and multi-compartment experiments.

Figure 9 shows the comparison between four setups: the pure biophysical model, the model with a single neural ODE augmentation, the adaptive *dt* version, and the two neural networks approach. In each case, I display the five best models (out of 300 trained in parallel for 300 epochs) based on their final losses. The base biophysical model reaches a best loss of 31, often failing to reproduce the spike amplitude. Introducing the neural ODE augmentation clearly improves performance, with all five top models outperforming the best base model. The main improvement is in capturing the spike height more accurately, although the detailed spike shape is still not reproduced perfectly. The adaptive *dt* strategy, on the other hand, performs no better than the base model (best loss around 31), while the two neural networks setup performs slightly better than the single neural ODE-augmented one, reaching a best loss of 13, with the top five models all lying between 13 and 18. Based on these results, I decided to use the 2-network framework for the following experiments.

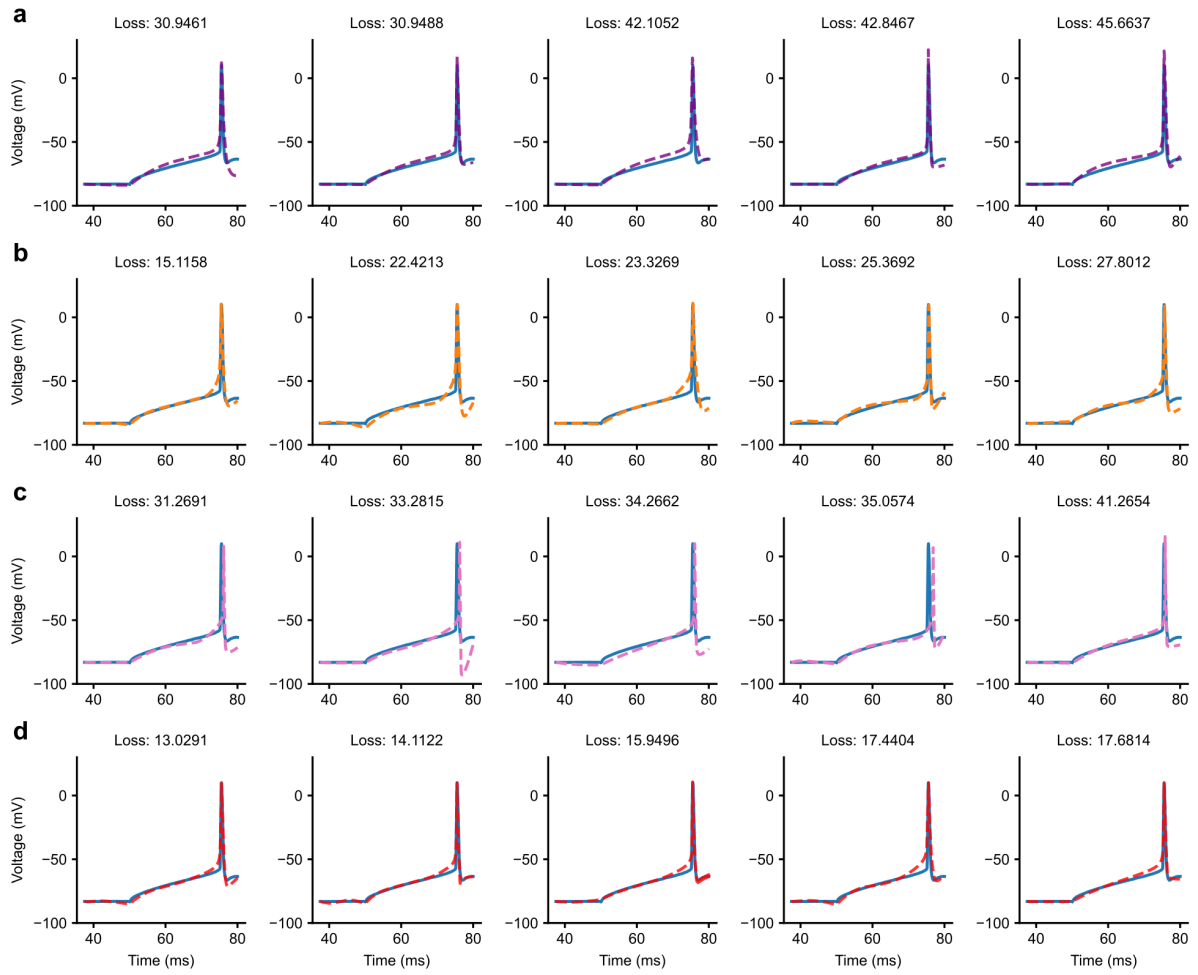

Figure 9: Comparison of model variants on the 80 ms experimental trace (neuron ID: 488683425) containing a single spike. The figure shows the five best models (out of 300 parallel trainings) for each setup based on the soft-DTW loss regulated with spike height. **Panel a:** Pure biophysical model. **Panel b:** Model with a single neural ODE augmentation. **Panel c:** Adaptive *dt* strategy. **Panel d:** Two neural networks approach.

To have a more detailed understanding of how the best-performing models behave internally, I analyzed the latent state u and the learned input current during the course of the simulation (Figure 10). The traces reveal that u evolves on a smaller scale than the voltage, acting as a kind of internal memory that governs how the additional current is injected. The learned additive current itself is highly structured, with sharp deflection during the spike that compensate for the limitations of the simple single-compartment dynamics. This confirms that the neural ODE-augmented system is not simply adding noise but is learning meaningful adjustments that systematically drive the model toward the target.

I also tested the extrapolation ability of these models by running them on slightly longer traces than those used for training. In this case, the performance quickly degrades, with spikes drifting out of alignment or being completely missed. This motivates the need for training on longer recordings, which is the focus of Section 6.2.3.

In addition, I examined the inferred ion channel parameters and the length values of the ten best models. Interestingly, they span a wide range rather than converging to a single biophysical solution. This diversity highlights the structure of the loss landscape: many different parameter–neural ODE combinations lead to qualitatively similar fits and comparable losses. In other words, the optimizer frequently settles in different local minima that are hard to distinguish based on short training traces alone.

Finally, I performed a small fine-tuning experiment by taking the 40 best models and continuing training with the simpler SSE loss instead of the DTW-based loss used initially. This adjustment provided a slight improvement in alignment, although the effect was modest overall. Together, these analyses suggest that while using two separate neural networks substantially improves the ability to capture spikes, the optimization problem remains highly non-convex and requires careful consideration of both model and training designs.

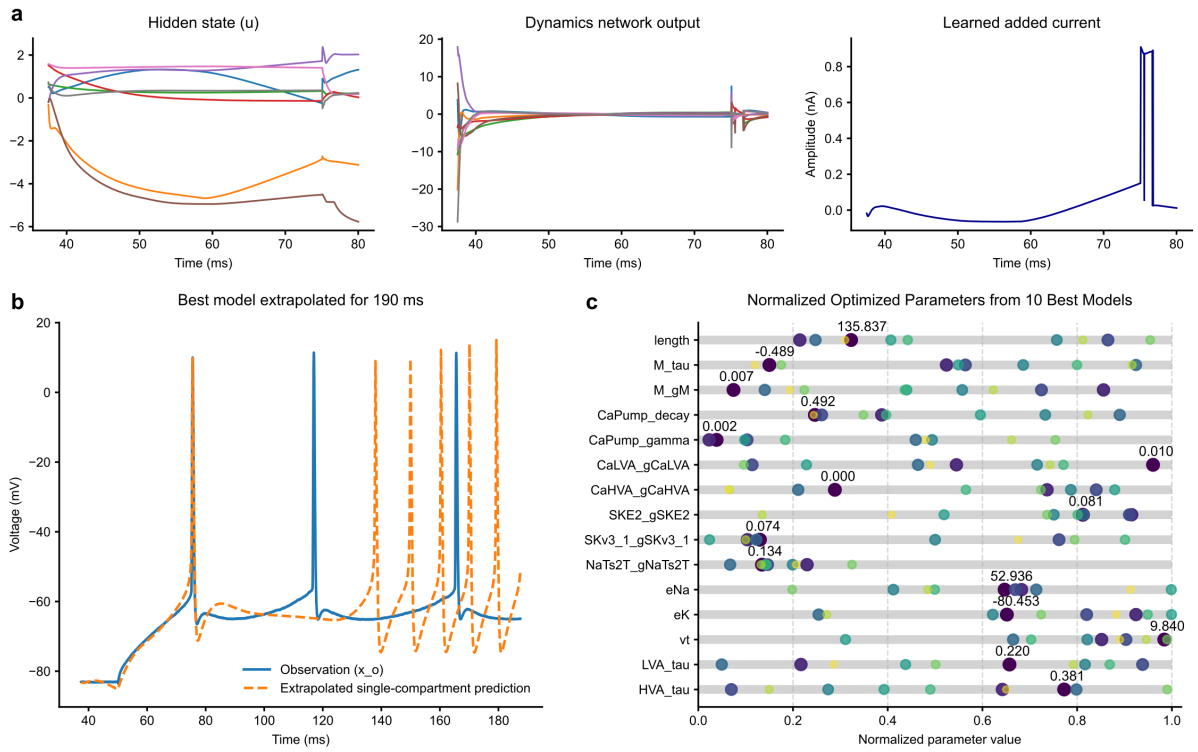

Figure 10: Details of the best model(s) after training with the modified architecture containing two separate neural networks. **Panel a:** Evolution of the hidden state u over the simulation, together with the output of the dynamics network and the learned added current produced by the readout network. **Panel b:** Extrapolation performance of a model trained on 80 ms of data when extended to 190 ms. After the first spike, the prediction becomes strongly misaligned with x_o. **Panel c:** Channel parameters and length values of the ten best models after training. Darker dots indicate models with lower loss; the exact parameter values of the best model are highlighted.

6.2.2 Computing the Optimal Added Current

To better understand the additive term that the neural networks should learn, I computed the optimal current making the single-compartment model exactly reproduce x_o. For this experiment, I focused on the first 250 ms of the recording (containing 4 spikes), allowing us to examine how the optimal current behaves in spiking versus non-spiking regions.

Analytic derivation. Starting from the passive membrane equation:

$$C_m \frac{\mathrm{d}V}{\mathrm{d}t} = -I_{\text{ion}} + I_{\text{ext}}, \tag{21}$$

the optimal current can be directly inferred from the mismatch in the voltage derivative. Since the membrane equation is expressed in terms of current densities (per unit area), the analytic expression for the optimal current must be multiplied by the membrane surface area to obtain the total current. For a target membrane potential V_{target} and a model prediction at the next time step V_{pred} with an external current of 0.0 nA, the required optimal current is:

$$I_{\text{opt}} = C_m \, A \, \frac{V_{\text{target}} - V_{\text{pred}}}{\Delta t}, \tag{22}$$

where A is the membrane surface area of the cylindrical compartment and C_m is the membrane capacitance that I fixed to 1.0 in my single-compartment models. This yields an analytic estimate of the optimal current at each time step.

Root search approach. In parallel, I implemented a binary root search procedure: at each time step I varied the injected current until the model's predicted voltage matched the target within a given tolerance. This method directly optimizes over the model's dynamics and avoids relying on the linearized derivative approximation.

Comparison of methods. Interestingly, the two approaches gave quite different results at first sight. The analytic method produced a trace with a soft-DTW loss of about 17, but required very high peak currents (up to $\sim$ 100 nA). The binary root search achieved an even lower loss (around 5), but we have to note that with this length of the data the difference between soft-DTW loss of 5 and 17 is not significant. However, this method found an even stronger optimal current, sometimes exceeding 200 nA. These results suggest two important conclusions. First, with random channel parameters, the optimal current at spike peaks is much larger than anticipated and far exceeds what the neural ODE-augmented model produced even in the best 2-network training setups. Second, it highlights the critical need to train the underlying channel parameters as well, since such large, sharp currents cannot be generated by a neural network governed by its own differential dynamics. To check how the optimal current looks like for a model with already trained channel parameters, I fitted the multi-compartment model to the same 250 ms dataset, and obtained a soft-DTW loss of 156. Computing the optimal current for this trained model revealed amplitudes in the range of [-10, 10] nA, much closer to the values produced by my previously trained neural ODE-augmented models (Figure 11).

One might ask whether this reduction could be an artifact of switching to a multi-compartment model. However, this is not the case. I also trained a single-compartment model, but with this setup its performance on the 250 ms dataset was poor: the best model produced only a single spike. This issue will be discussed in more detail in Section 6.2.3. The corresponding optimal current was therefore only slightly lower than the results from the untrained single-compartment model (first two columns of Figure 11).

In contrast, the best trained multi-compartment model already produced four spikes with the neural-ODE augmented system. Consequently, the required additive current was much smaller, as the network only needed to adjust spike timing and amplitude rather than generate spikes outright. This underlines why training the channel parameters is essential: if the model fails to produce spikes on its own, the added corrective current must be unrealistically large to compensate. But if the model already produces spikes, it only requires added currents an order of magnitude smaller to fine-tune timing and amplitude.

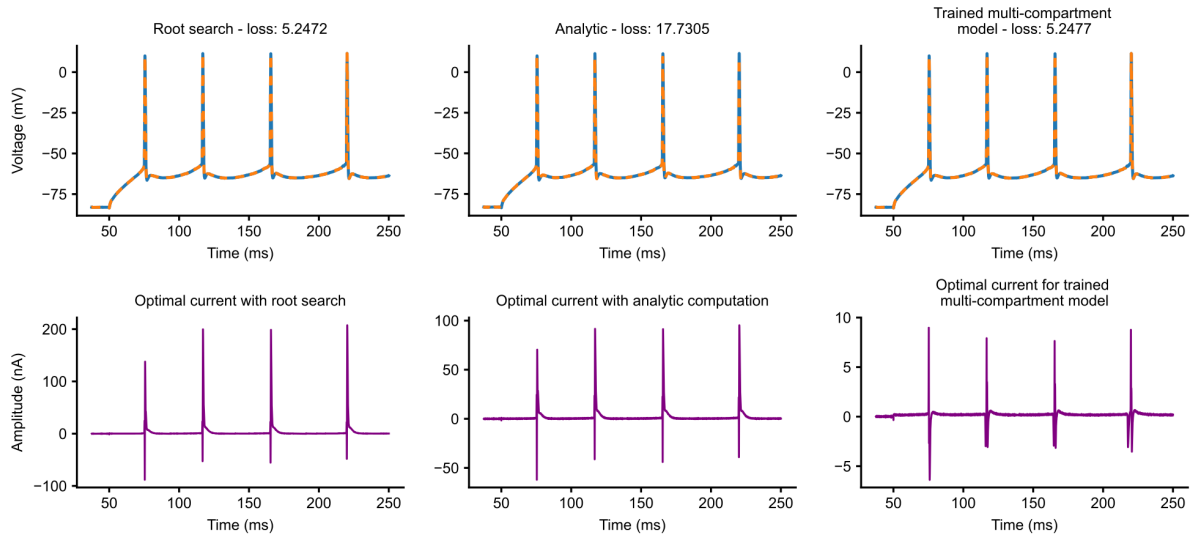

Figure 11: Comparison of optimal currents and corresponding voltage traces. **First column:** Blue solid line shows the target experimental voltage trace (x_o), while dashed orange line represents the prediction using the optimal current found by root search. Below, the purple trace shows the optimal current found and applied. **Second column:** Same as the first column, but using the analytically computed optimal current. **Third column:** Optimal current required for the trained multi-compartment model; note that the range of the added current is much smaller compared to the other methods, which makes it easier to be learned by the neural ODE-augmented system.

Channel-level decomposition. Having computed the optimal additive current, I next explored how this current is distributed relative to the main ionic channels in the model. I extracted the contributions of sodium, potassium, leak, calcium, and H-current from the state variables:

$$I_{\mathrm{Na}}, \quad I_{\mathrm{K}}, \quad I_{\mathrm{Leak}}, \quad I_{\mathrm{Ca}}, \quad I_{\mathrm{H}}.$$

Plotting these over time shows that the sodium current dominates during the rising phase of the spike, while potassium provides the main opposing current in the falling phase (Figure 12). The leak, calcium, and H-currents are comparatively small but contribute to modulating the baseline and slightly shaping the spike waveform.

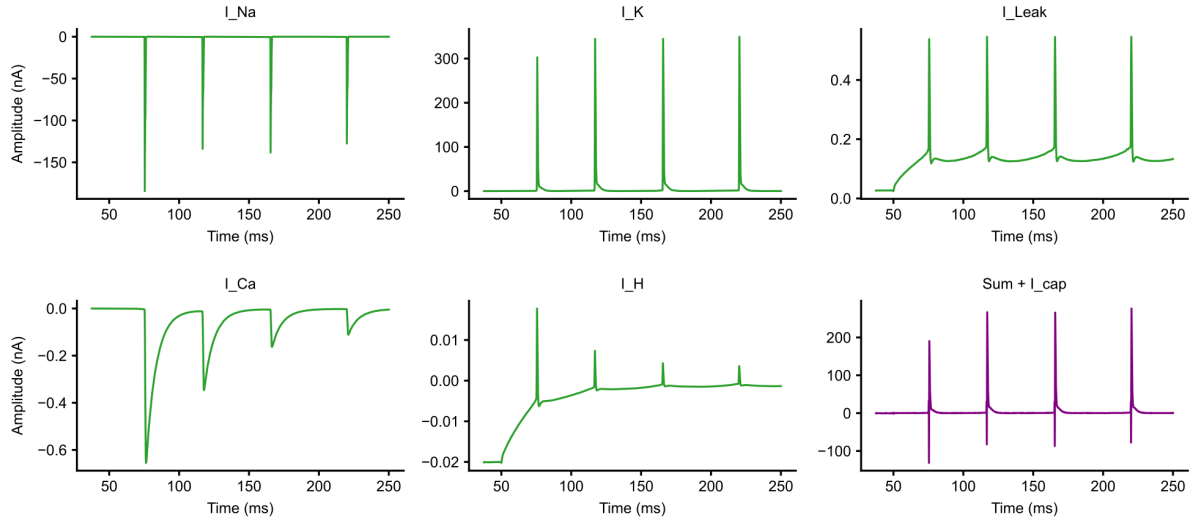

Figure 12: Distribution of the optimal external current among the main ionic channels in green: sodium, potassium, leak, calcium, and H-current. In purple, the sum of I_{Na}, I_{K}, I_{Leak}, I_{Ca}, and I_{H} completed by the capacitive term.

A summed trace of all channel currents, when combined with the capacitive current, should in principle match the optimal external input required to exactly align the membrane potential with the target trace:

$$I_{\text{ext}} = C_m \frac{dV}{dt} + I_{\text{ion}} \tag{23}$$

In practice, they do not match perfectly due to nonlinear interactions, numerical discretizations in the step function, channel gating dynamics, and the limited time resolution of the simulation. Although the exact magnitudes differ, the qualitative agreement still helps interpret where and with what order of amplitude the added signal would need to enter the system. This analysis illustrates that, while the spike is primarily driven by sodium influx and repolarized by potassium, the other channels modulate the baseline and slightly shape the spike waveform.

Summary. These experiments show that reproducing precise spike waveforms requires additive currents that are both large and temporally precisely localized, far exceeding what the neural ODE-augmented framework alone can generate with untrained channel parameters. The channel-level decomposition highlights that sodium and potassium currents dominate the rapid voltage changes during spikes, while other channels fine-tune the baseline and spike shape. Together, these results emphasize the necessity of jointly optimizing both the neural ODE and the underlying biophysical channel parameters to achieve accurate single-compartment model fitting.

6.2.3 Fitting to Longer Voltage Traces

After experimenting with different setups for fitting the single-compartment model to x_o containing a single spike, and identifying the architecture consisting of two separate neural networks as the most effective, I next evaluated its ability to reproduce longer voltage traces with multiple spikes. Specifically, I extended the training window from 80 ms (single spike) to 150 ms, which included two consecutive spikes.

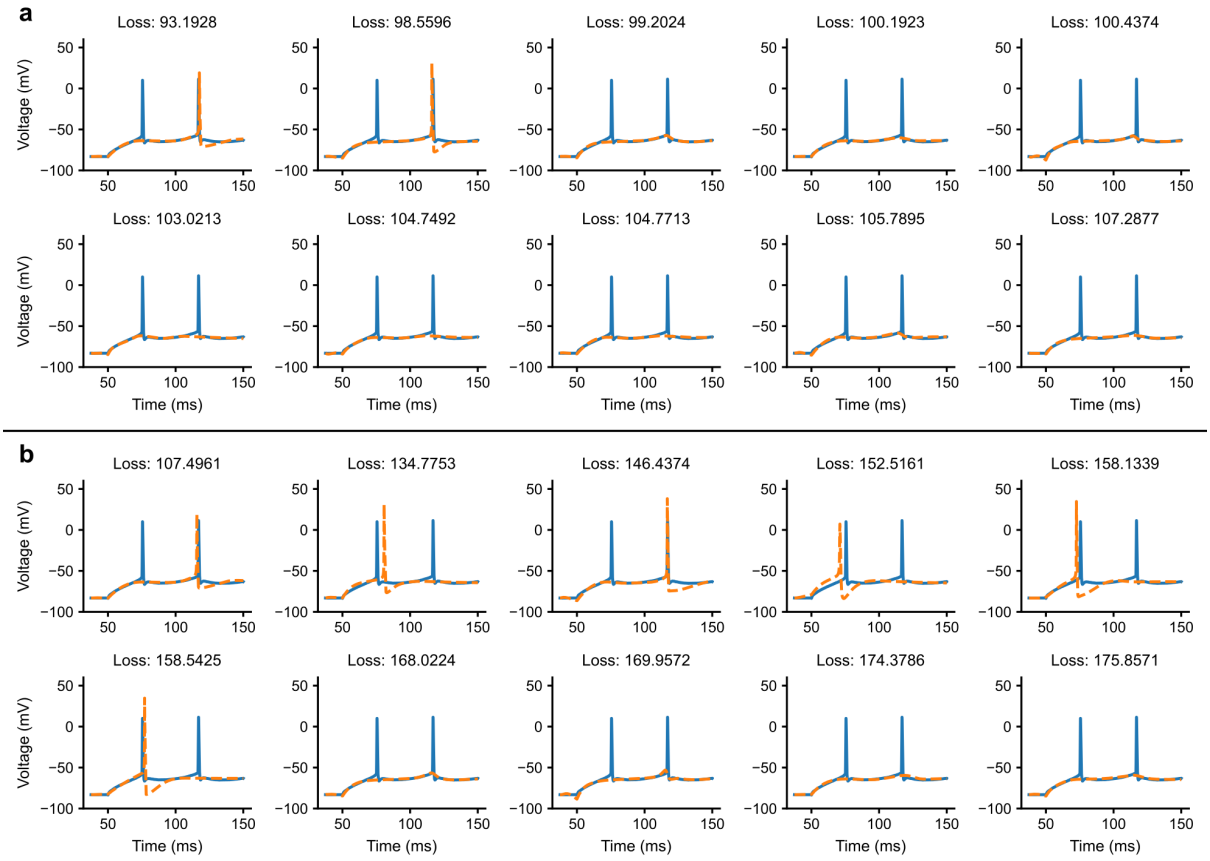

Figure 13: Ten best results after training 300 models in parallel with the 2-network setup to fit x_o of 150 ms. **Panel a:** Using the pure soft-DTW loss. **Panel b:** Using the regularized soft-DTW loss.

The results with the 2-network framework using the pure soft-DTW loss are rather disappointing: only the two best predicted voltage traces exhibit spikes (and even then, only one), while the other responses remain completely flat (panel a of Figure 13). This suggests that the no-spike solutions correspond to very flat but frequent local minima: once a model is initialized or driven into such a regime, it becomes difficult to escape. Interestingly, traces with two or three spikes that are slightly misaligned in timing and amplitude appear more plausible to the human eye than the flat response, yet the soft-DTW loss often rates the flat solution as better. This discrepancy raises the question of how models behave at initialization: if most start far from the ground truth (e.g., typically failing to spike at all), it may indicate that modifications to the biophysical model itself are needed.

Before analyzing the traces generated by the models at initialization, I tested whether adding a small regularization term to the soft-DTW loss could improve performance. Following the approach in Section 6.2.1, I used the absolute difference between the maximum values of the simulated trace and x_o. This modification achieved a modest improvement—the best six predicted traces now contained spikes (panel b of Figure 13), but it also became clear that such a simple regularization cannot substantially overcome the limitations observed in the longer-trace setting.

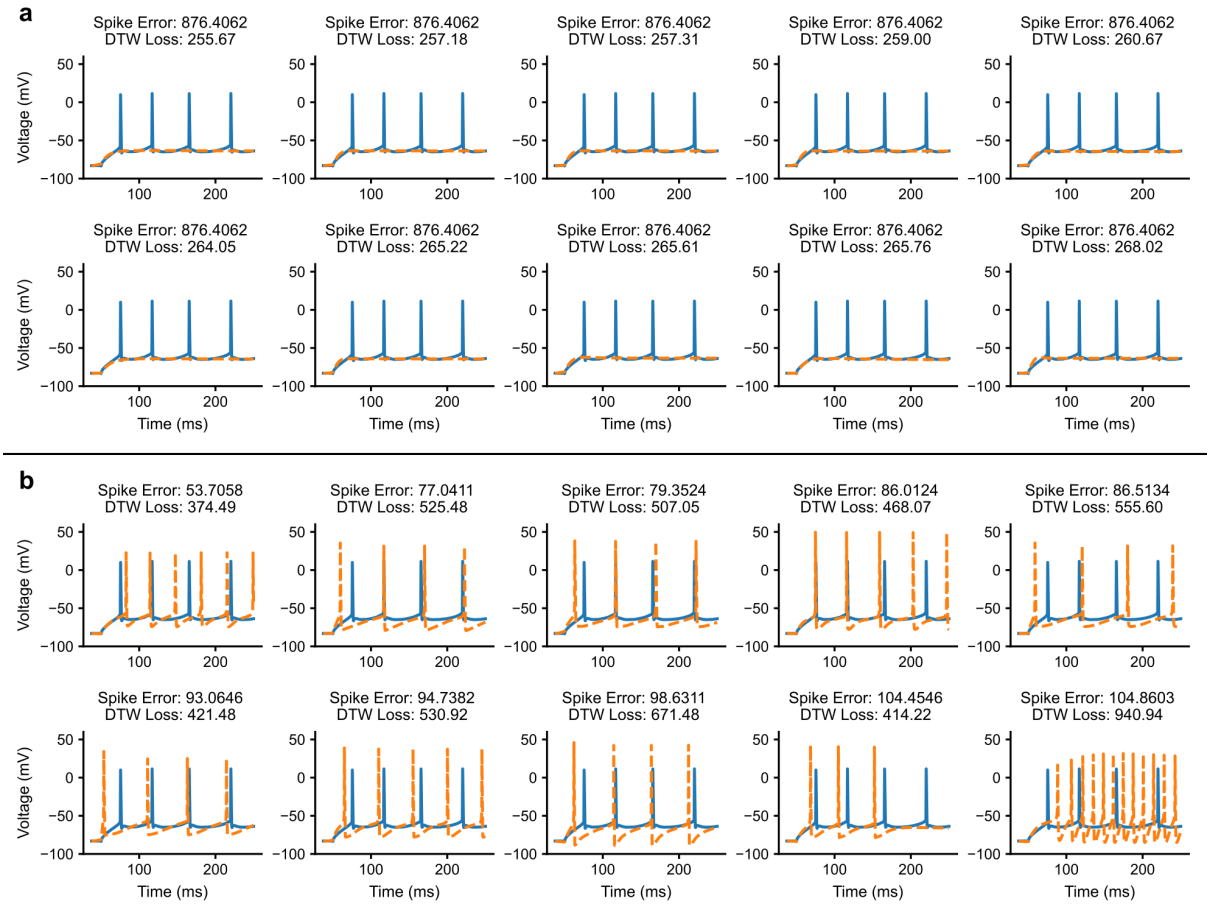

Figure 14: Comparison of the top ten randomly initialized full single-compartment models (out of 3000) evaluated with two different loss functions. **Panel a:** Traces ranked according to the pure soft-DTW loss. Here, the top models are exclusively flat, illustrating that soft-DTW favors non-spiking traces even when they are qualitatively less similar to x_o. **Panel b:** Traces ranked according to a hand-crafted statistical loss based on spike count, spike timing, and spike amplitudes. In this measure, the best models always contain spikes.

To test these limitations more systematically, I compared the behavior of different base models at initialization, without any training or using the neural ODE. Alongside the full single-compartment model used so far, I also tried reduced versions with successive ion channel types removed: first without H-channels, then without both H- and M-channels, and finally without H-, M-, and Ca-channels. For comparison, I also included the full multi-compartment model. In each case, I evaluated 3000 random initializations compared to the experimental data of 250 ms. A clear pattern emerged: the fewer channel types present,

the more non-flat traces were produced (between 450 and 1150 out of 3000). However, across all single-compartment variants, the top models according to the soft-DTW loss were exclusively flat: none of the best ten traces contained any spikes (panel a of Figure 14). In contrast, the multi-compartment model proved more promising, with 10 spiking responses among the top 20 (Figure 15). This explains why longer-trace training with single-compartment models so consistently collapsed to flat predictions: the best models after random initialization all come from a very flat region of the loss landscape (where they give flat voltage responses), making it difficult for the optimizer to escape and produce spiking behavior during training.

To better understand this discrepancy, I also assessed the same traces with a hand-crafted statistical loss based on spike count, amplitudes, and spike timings (panel b of Figure 14). Unlike soft-DTW, this measure reflects our intuitive notion of similarity: here, the best models always contained spikes, although often misaligned in time or amplitude. This contrast shows that the problem lies not only in the large fraction of flat traces at initialization (around two-thirds to five-sixths, depending on the model), but also in the fact that the spiking responses that do emerge are judged too dissimilar to the data under the soft-DTW distance. Since the multi-compartment model produced more encouraging results, and since morphology should in principle support richer dynamics, the next step was to attempt fitting the 150 ms recording with the biophysically more detailed multi-compartment model.

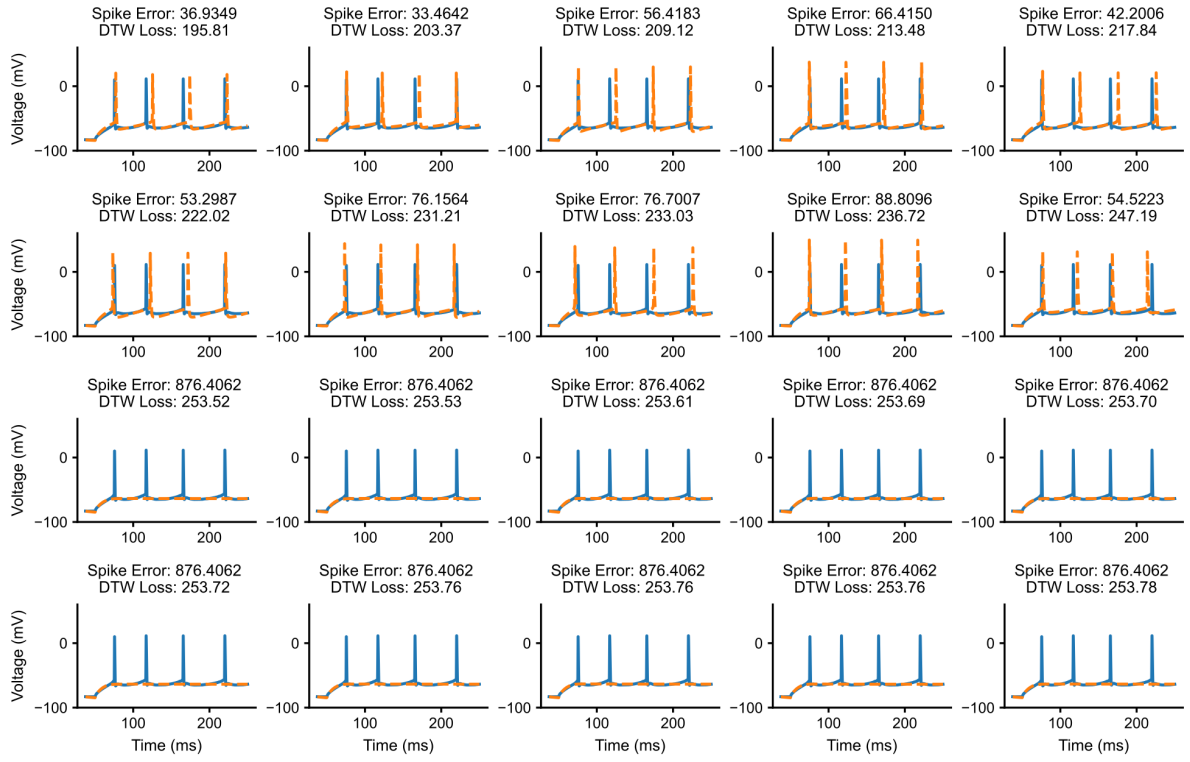

Figure 15: Comparison of the top 20 randomly initialized multi-compartment models (out of 3000) evaluated with the pure soft-DTW loss. The top ten contain spikes, the others remain completely flat.

6.3 Multi-Compartment Model Fitting

Given the limitations of the single-compartment setup, the natural next step was to test whether adding morphological detail could improve the fit to longer experimental recordings. To this end, I repeated the training procedure using a multi-compartment neuron model on the same 150 ms trace. The expectation was that the richer dynamics of the multi-compartment structure would provide traces closer to x_o both at initialization and after training, and help overcome the collapse to flat predictions observed in the single-compartment case.

The results, however, show that simply moving to a multi-compartment model does not resolve the main difficulties. Even after experimenting with different training setups, the best outcomes remained

comparable to the single-compartment case. Among the best ten models, six produced two spikes and two produced one spike, but the overall alignment in amplitude and timing was still poor (Figure 16). Notably, two flat models were also among the top ten, which highlights that out of the 300 parallel trainings, only a handful of models (eight in this case) actually achieved a lower soft-DTW loss than the optimized flat solutions. Extending the setup further to the 250 ms recording with four spikes only worsened performance: depending on the hyperparameters, only one to six of the ten best models contained any spikes (Figure 17). This confirms the earlier observation that the longer the trace and the more spikes it contains, the harder it becomes to avoid or escape the flat regions of the loss landscape and to reproduce responses with lower soft-DTW loss than non-spiking traces.

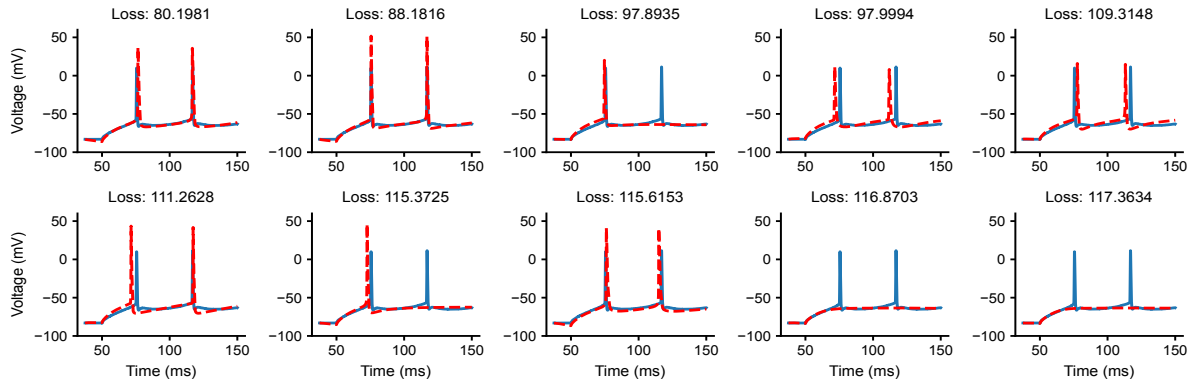

Figure 16: Voltage traces from the ten best multi-compartment models fitted to x_o of 150 ms. Two models out of the ten best produce no spike at all.

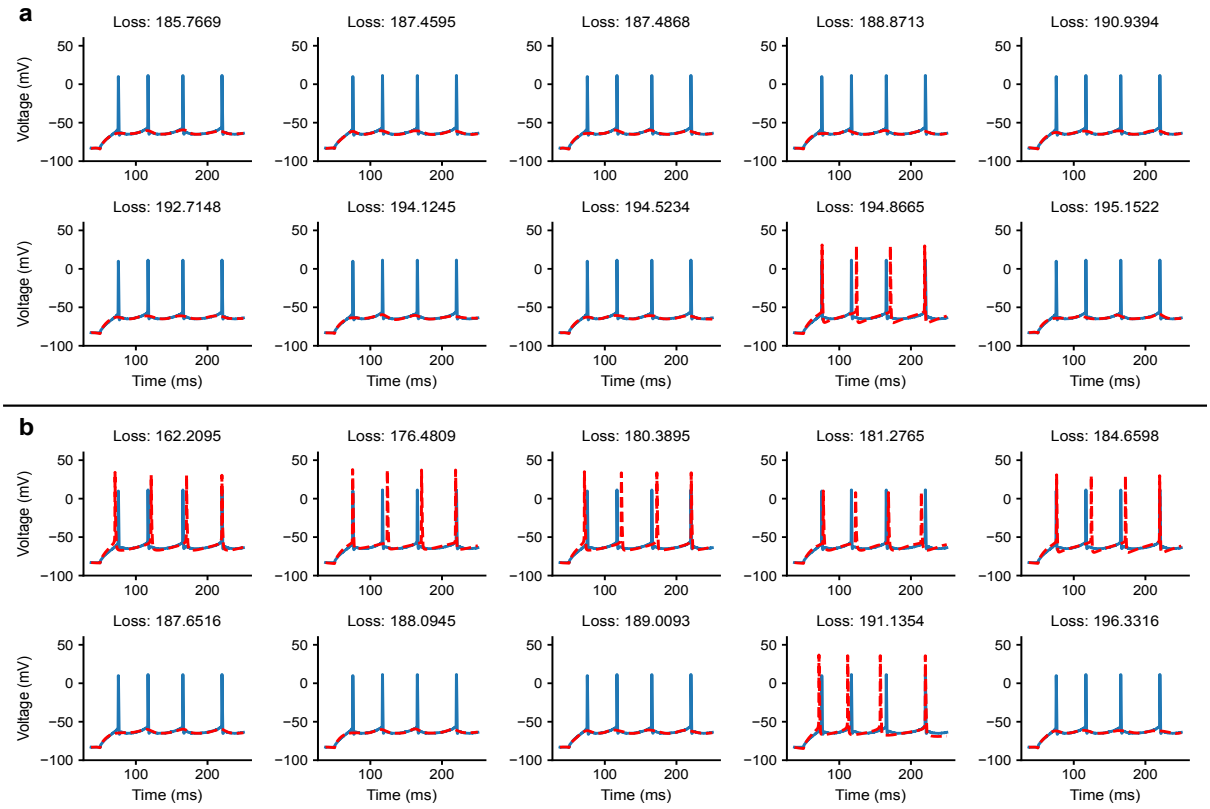

Figure 17: Voltage traces from the ten best multi-compartment models fitted to x_o of 250 ms. **Panel a:** Results with the added current's standard deviation being 0.1 and the neural network parameters clipped to have a norm of maximum 10.0. **Panel b:** Results with the added current's standard deviation being 0.01 and the neural network parameters clipped to have a norm of maximum 100.0.

To better understand these outcomes, I also inspected the parameters of the ten best multi-compartment models for both the 150 ms and 250 ms cases. As in the single-compartment experiments (Figure 10), the resulting parameter sets exhibited substantial diversity, with no clear convergence toward a common solution. This further supports the idea that the challenge lies less in morphological expressivity and more in the biophysical construction of the models and the structure of the optimization landscape.

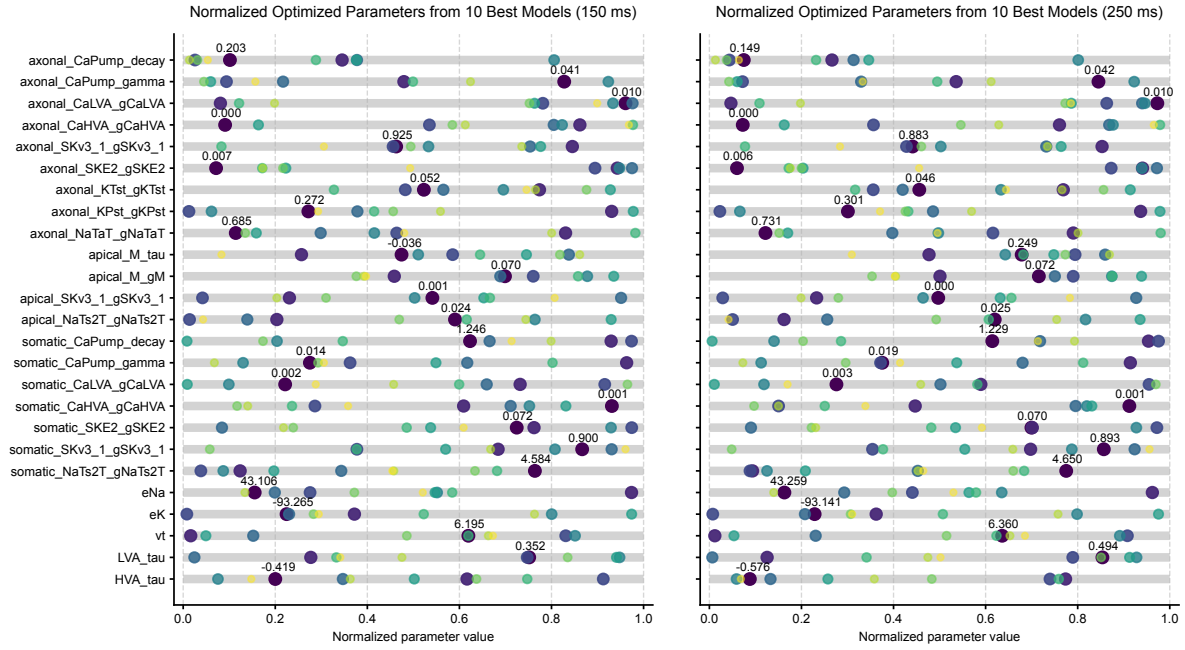

Figure 18: Comparison of the trained parameters of the ten best multi-compartment models fitted to x_o of 150 ms and 250 ms. The results reveal a similar diversity of parameter values as observed in the single-compartment case, with no clear convergence across models.

At the same time, training multi-compartment models is significantly more computationally demanding, which makes them impractical for large-scale parallel searches. Since the ultimate goal of this work is to use neural ODEs to bridge the gap between single- and multi-compartment (or real) neuron dynamics, I chose to focus subsequent experiments on optimizing the single-compartment framework. In particular, I explored modifications to its biophysical structure while keeping the multi-compartment results as a reference point.

7 Alternative Channel Models

7.1 Motivation and Setup

Up to this point, my experiments focused primarily on simplified active models with a limited set of Markram-type channels, testing whether a neural ODE–based augmentation can improve their fit to x_o. While this provided valuable insights, these models remain highly constrained in the kinds of dynamics they can capture by the specific channel types used. Real neurons, however, express a wider variety of ion channels with diverse kinetics, so the introduction of new types of channels may improve both the richness and the realism of the simulated dynamics.

In this section, I explore the use of Pospischil-type channels (Pospischil et al., 2008), as they offer a biophysically realistic set of Hodgkin–Huxley-type ion channels commonly used to model cortical neurons. These include, in addition to the standard sodium, potassium, and leak channels, low-threshold calcium (CaT) and high-threshold calcium (CaL) channels, among others. The motivation for this exploration is not that these channels are inherently more complex or capable than the Markram-type set, but that they offer a complementary formulation of ion channel dynamics, including slightly different voltage dependencies and gating kinetics. This allows us to test whether the choice of channel models affects the ability of the neuron to reproduce x_o, both at initialization and after training, both with and without the neural ODE augmentation.

To systematically investigate these effects, I set up a series of experiments in which I vary the channel composition and gating parameters. I train the neuron models on short and long voltage traces containing one or multiple spikes, both with and without the neural ODE framework that augments the base model. Additionally, I experiment with learnable gating parameters to allow the model to adjust gating curves smoothly between discrete reference points, providing another layer of flexibility. I also attempt to make the models learn the exponents of the activation and inactivation gates (m, h, and n) directly. These experiments are designed to answer several questions: how the choice of channel types influences the initial model behavior, whether it improves training convergence or the ability to reproduce spike timing, and how well the trained models generalize to longer voltage recordings.

In the following subsections, after discussing the Pospischil-type channels in more detail, I first present results using this channel set under different training conditions, and then turn to the experiments with modified gating functions and exponents.

7.2 Pospischil-Type Channels

In this part of the project, I experiment with Pospischil-type channels (Pospischil et al., 2008), a set of widely used Hodgkin–Huxley-style voltage-dependent channels for modeling cortical and hippocampal pyramidal cells. These models were originally taken from the literature and represent one possible choice among many alternatives. Some of their parameter values were previously adjusted to fit voltage-clamp recordings, but no effort was made to specifically calibrate them for the preparation used in the original paper, nor for the experimental data in this thesis. My goal is to evaluate how these channels perform compared to the previously used Markram-type channels, and how well the model can learn adjustments to their kinetics via interpolated parameter modifications.

Sodium and potassium currents for action potentials. To generate action potentials, two fundamental types of voltage-gated currents are required: a fast inward sodium current and a delayed outward potassium current. Both of these are modeled as modified Hodgkin–Huxley-type mechanisms adapted for central neurons based on Traub and Miles (1991), and together they capture the essential depolarizing and repolarizing phases of the spike.
The voltage-dependent sodium current is modeled as a modified Hodgkin–Huxley current, described by the following equations:

$$
\begin{aligned}
I_{\mathrm{Na}} &= \bar{g}_{\mathrm{Na}}\, m^3 h\, (V - E_{\mathrm{Na}}), \\
\frac{\mathrm{d}m}{\mathrm{d}t} &= \alpha_m(V)(1 - m) - \beta_m(V)m, \\
\frac{\mathrm{d}h}{\mathrm{d}t} &= \alpha_h(V)(1 - h) - \beta_h(V)h,
\end{aligned}
$$

where the opening and closing transition rate functions are given by:

$$\alpha_m = \frac{-0.32(V - V_T - 13)}{\exp[-(V - V_T - 13)/4] - 1}, \qquad \alpha_h = 0.128 \exp[-(V - V_T - 17)/18],$$
$$\beta_m = \frac{0.28(V - V_T - 40)}{\exp[(V - V_T - 40)/5] - 1}, \qquad \beta_h = \frac{4}{1 + \exp[-(V - V_T - 40)/5]},$$

and the variable V_T adjusts spike threshold.
The delayed-rectifier potassium current is defined as:

$$I_{\mathrm{K}} = \bar{g}_{\mathrm{K}}\, n^4\, (V - E_{\mathrm{K}}),$$
$$\frac{\mathrm{d}n}{\mathrm{d}t} = \alpha_n(V)(1 - n) - \beta_n(V)n,$$
$$\alpha_n = \frac{-0.032\,(V - V_T - 15)}{\exp[-(V - V_T - 15)/5] - 1},$$
$$\beta_n = 0.5\, \exp[-(V - V_T - 10)/40].$$

In my model, the maximal conductances $\bar{g}_{\mathrm{Na}}$ and $\bar{g}_{\mathrm{K}}$, the reversal potentials E_{Na} and E_{K}, and the threshold variable V_T are treated as learnable parameters, while the gating kinetics (α and β functions of the m, h, and n gates) are now fixed. In later experiments, I extend this framework by introducing interpolated modifications to the α and β rate functions to test how well the model can adjust channel kinetics.

Other voltage-dependent currents. In addition to the sodium and potassium currents required for spike generation, other ionic currents are incorporated to capture additional neural dynamics such as spike adaptation and afterhyperpolarization.

For spike-frequency adaptation, a slow non-inactivating potassium current (I_{Km}) is included based on Yamada et al. (1989):

$$I_{Km} = \bar{g}_{Km}\, p\, (V - E_K), \qquad p_\infty(V) = \frac{1}{1 + \exp\big[-(V + 35)/10\big]},$$
$$\frac{\mathrm{d}p}{\mathrm{d}t} = \frac{p_\infty(V) - p}{\tau_p(V)}, \qquad \tau_p(V) = \frac{\tau_{\max}}{3.3 \exp\big[(V + 35)/20\big] + \exp\big[-(V + 35)/20\big]}.$$

For bursting behavior, high-threshold and low-threshold calcium currents are incorporated. The high-threshold calcium current ($I_{\mathrm{Ca_L}}$) is described by Reuveni et al. (1993) as:

$$I_{\mathrm{Ca_L}} = \bar{g}_{\mathrm{Ca_L}}\, q^2 r\, (V - E_{\mathrm{Ca}}), \qquad \frac{\mathrm{d}q}{\mathrm{d}t} = \alpha_q(V)(1 - q) - \beta_q(V)q, \qquad \frac{\mathrm{d}r}{\mathrm{d}t} = \alpha_r(V)(1 - r) - \beta_r(V)r,$$

while the low-threshold calcium current ($I_{\mathrm{Ca_T}}$) described by Destexhe et al. (1996a) and Destexhe et al. (1996b) is expressed as:

$$I_{\mathrm{Ca_T}} = \bar{g}_{\mathrm{Ca_T}}\, s_\infty^2 u\, (V - E_{\mathrm{Ca}}), \qquad \frac{\mathrm{d}u}{\mathrm{d}t} = \frac{u_\infty(V) - u}{\tau_u(V)},$$

where the activation variable s is assumed to be at steady state due to its fast activation dynamics. These currents are responsible for generating different types of bursting and rebound behaviors. The detailed expressions for $s_\infty(V)$, $u_\infty(V)$, $\tau_u(V)$, as well as α_q, β_q, α_r, and β_r can be found in the original paper (Pospischil et al., 2008). Similarly to the channels described above, my model treats the maximal conductances, as well as the parameters $\tau_{\max}$ and E_{Ca} as trainable during optimization.

Motivation for gating dynamics modifications. Each gating variable in these channels depends on voltage through transition rate functions, such as $\alpha(V)$ and $\beta(V)$ in sodium and potassium channels. To allow the neuron model to flexibly match x_o, I later modify these rate functions by adding small interpolated values between discrete points in the voltage domain, making these points learnable. I also experiment with treating the exponents of the m, h, and n gates as trainable parameters. This approach will allow the model to learn fine-grained adjustments to the gating kinetics while preserving the overall structure of the original Pospischil-type channels.

7.3 Experiments with Standard Pospischil-Type Channels

In the first set of experiments, I investigated how well the single-compartment neuron model could reproduce x_o using only the standard Pospischil-type channels described above, without modifying the gating dynamics or the exponents of the gating variables. The default model included the leak, Na, and K channels. Additional channels—CaT, CaL, and Km—were incrementally added to assess their contribution to matching x_o. Based on the results presented below, the CaT and Km channels were retained for further analysis, as they improved the fit for at least some of the examined cells, whereas the CaL channel had little effect.

The model parameters optimized in these experiments included not only the maximal conductances ($\bar{g}$) of each channel but also other key biophysical parameters: the reversal potentials ($E_{\text{Na}}, E_{\text{K}}, E_{\text{Ca}}$), the spike threshold shift (V_T), the compartment length, and the $\tau_{\max}$ parameter of the I_{Km} current. Bounds for each parameter were set to physiologically plausible ranges, for example:

- $\bar{g}_{\text{Na}} \in [5.0 \times 10^{-3},\ 5.0 \times 10^{-2}]$ S/cm^2
- $\bar{g}_{\text{K}} \in [1.0 \times 10^{-4},\ 1.5 \times 10^{-2}]$ S/cm^2
- $\bar{g}_{\text{Km}} \in [2.0 \times 10^{-6},\ 6.0 \times 10^{-4}]$ S/cm^2
- length $\in [100, 400]$ µm

For each configuration, I analyzed both shorter traces with one or two spikes and longer traces with three or four spikes. Optimization was performed with and without the neural ODE augmentation to evaluate its impact on fitting performance. All experiments were conducted across four different cells, and results were examined at initialization and after training.

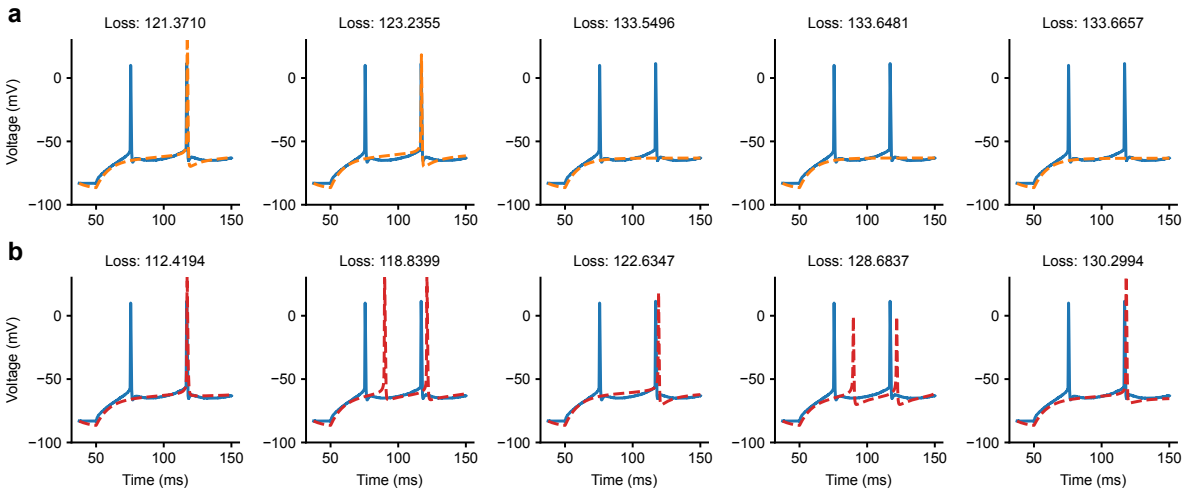

Figure 19: Best five voltage traces out of 3000 random parameter initializations obtained with a single-compartment model using Pospischil-type channels for cell 488683425. The fits were selected based on the soft-DTW loss over 150 ms of data. **Panel a:** Model including only leak, Na, and K channels. **Panel b:** Model including additional CaT and Km channels.

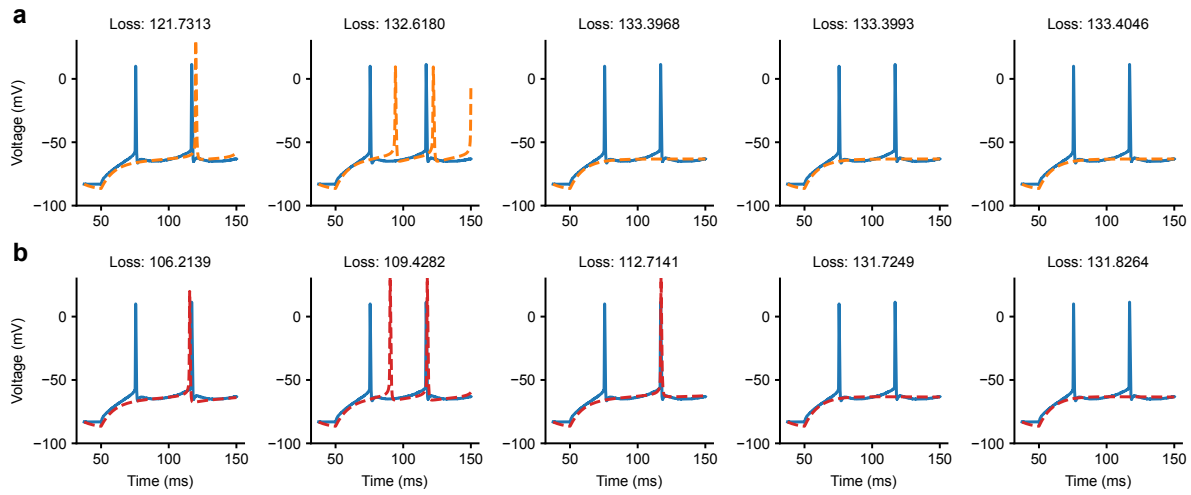

Figure 20: Best five voltage traces after training the base single-compartment model using Pospischil-type channels for cell 488683425. The fits were selected based on the soft-DTW loss over 150 ms of data. **Panel a:** Model including only leak, Na, and K channels. **Panel b:** Model including additional CaT and Km channels.

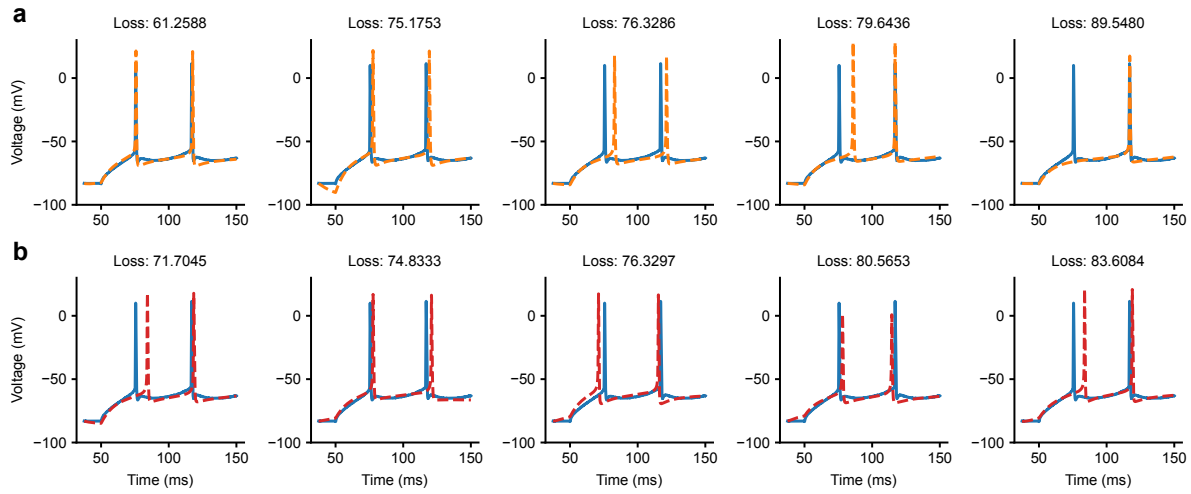

Figure 21: Best five voltage traces after training the neural ODE-augmented single-compartment model using Pospischil-type channels for cell 488683425. The fits were selected based on the soft-DTW loss over 150 ms of data. **Panel a:** Model including only leak, Na, and K channels. **Panel b:** Model including additional CaT and Km channels.

The four figures Figure 19, Figure 20, Figure 21, Figure 22 illustrate the effect of adding CaT and Km channels to the default Pospischil-type model (only including leak, Na, and K channels). While the improvement in fit quality is not dramatic, the results do show a modest benefit from including these additional channels, both at initialization and after training. More importantly, two broader findings emerge. First, the neural ODE augmentation proves essential: after training, it consistently reduces the loss by 30-50% compared to the base model. Second, the use of Pospischil-type channels substantially improves the model's ability to reproduce spiking behavior. In earlier experiments with Markram-type channels, only 2 of the 10 best models exhibited any spiking activity, and none matched the two spikes observed in x_o (Figure 13). In contrast, in the present experiments all of the top models generated spiking, with the configuration using CaT and Km channels reliably producing the correct number of spikes as well. This demonstrates that modifying the underlying biophysical model by adopting a different set of channels is a worthwhile direction, as it leads to significantly better results. This also suggests that further refinements in the choice of biophysical mechanisms could yield even more accurate models.

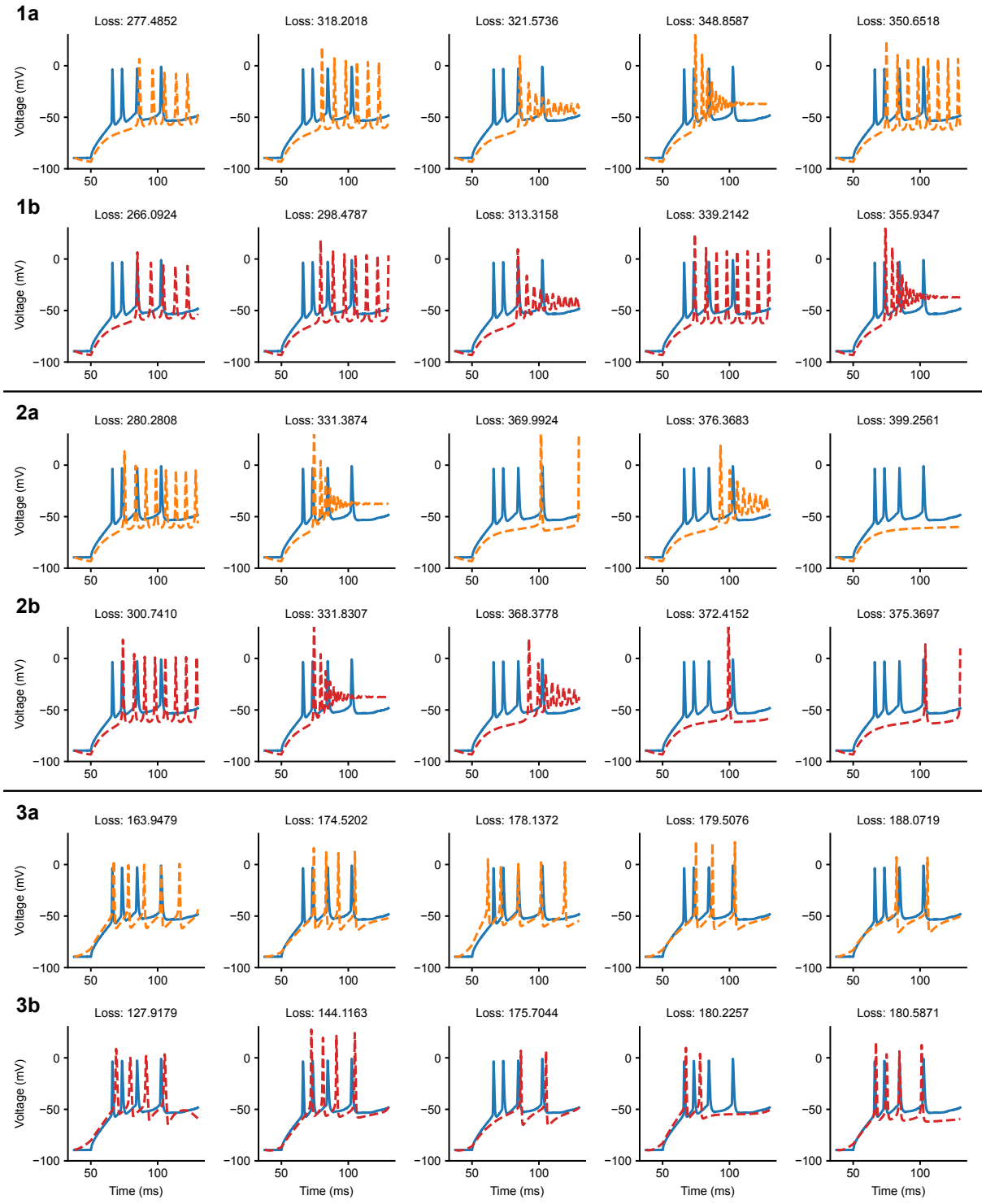

Figure 22: Results of the same experimental setup as in Figure 19, Figure 20, and Figure 21, now applied to cell 473601979 over a 130 ms recording. Compared to the previous cell, this neuron exhibits less regular spiking: it produces four spikes within this window, with inter-spike intervals that increase over time. This makes x_o considerably more challenging to fit. Nevertheless, the benefits of including additional CaT and Km channels (red traces) as well as augmenting the model with a neural ODE are clearly visible. **Panel 1:** Best five voltage traces out of 3000 random parameter initializations. **Panel 2:** Best five voltage traces after training the base single-compartment model. **Panel 3:** Best five voltage traces after training the neural ODE-augmented single-compartment model.

7.4 Experiments with Modified Gating Dynamics and Learnable Exponents

In the next set of experiments, I extended the Pospischil-type channel formulations by allowing small, learnable modifications to the gating dynamics and by relaxing the fixed integer exponents of the gating variables. It is worth noting that the commonly used gating functions and exponents are not exact ground truth but empirical estimates based on specific experimental conditions, and considerable variability across preparations has been observed (Marder and Taylor, 2011). Specifically, for the Na, K, and CaT channels I introduced an additional interpolated term into their transition rate functions governing activation and inactivation dynamics. The interpolation was defined by a small set of learnable points, enabling the model to flexibly adjust the shape of the gating curves during training while preserving their overall structure.

In addition to modifying the gating kinetics, I also allowed the powers of the gating variables (m, h, and n) to vary within restricted ranges around their standard integer values. In practice, rather than fixing these exponents to canonical values (m^3h for Na or n^4 for K), they were treated as learnable parameters, constrained to remain close to physiologically plausible ranges. This approach was motivated by the observation that even small deviations in gating exponents can strongly influence current dynamics, thereby potentially improving the model's ability to capture cell-specific firing behavior.

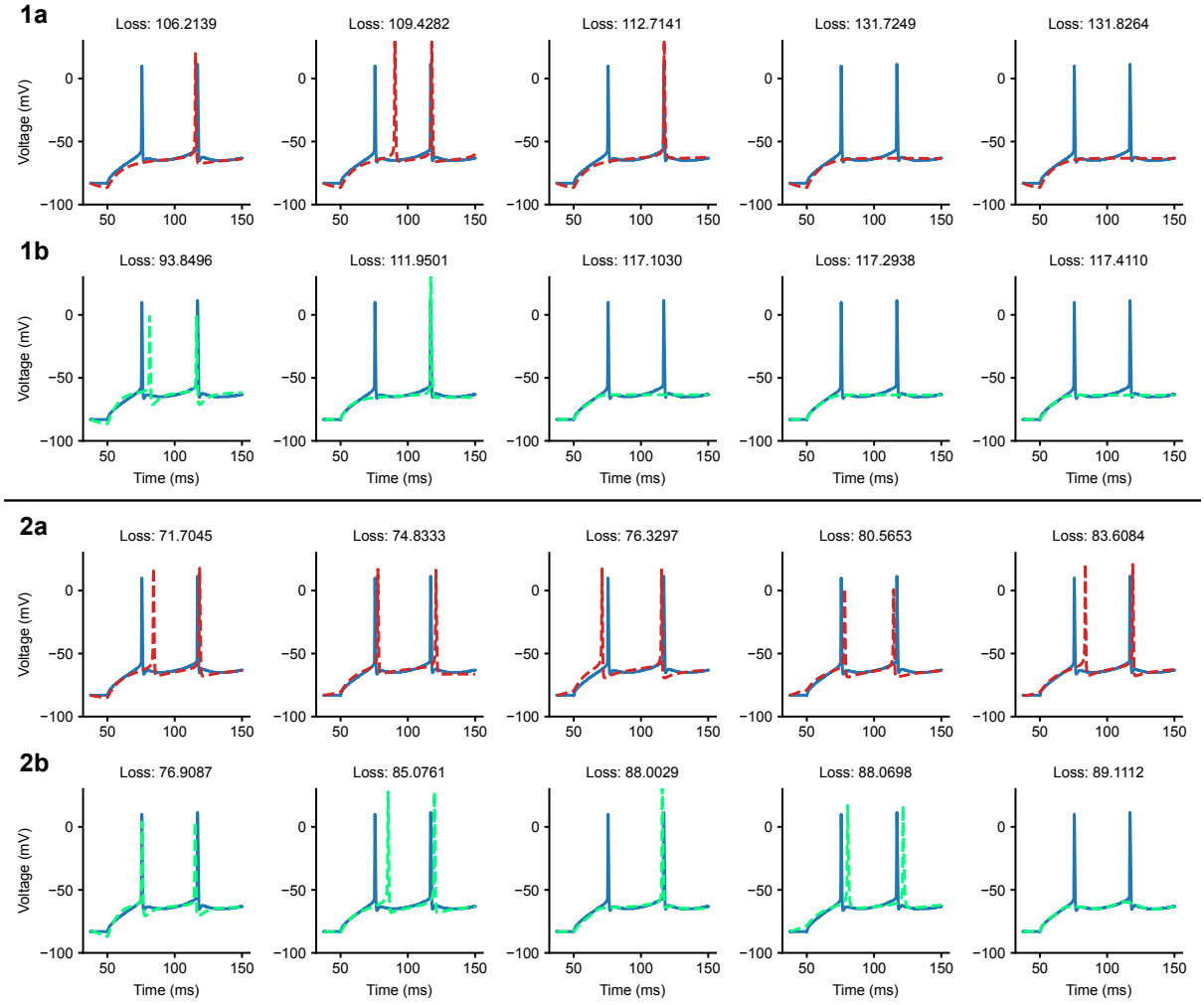

Figure 23: Top five voltage traces after training the single-compartment model with Pospischil-type channels (leak, Na, K, CaT, and Km) for cell 488683425. Fits were selected based on the soft-DTW loss over 150 ms of x_o. **Panel 1a:** Base model with standard channels. **Panel 1b:** Base model with learnable gating functions and exponent modifications. **Panel 2a:** Neural ODE-augmented model with standard channels. **Panel 2b:** Neural ODE-augmented model with learnable gating functions and exponent modifications.

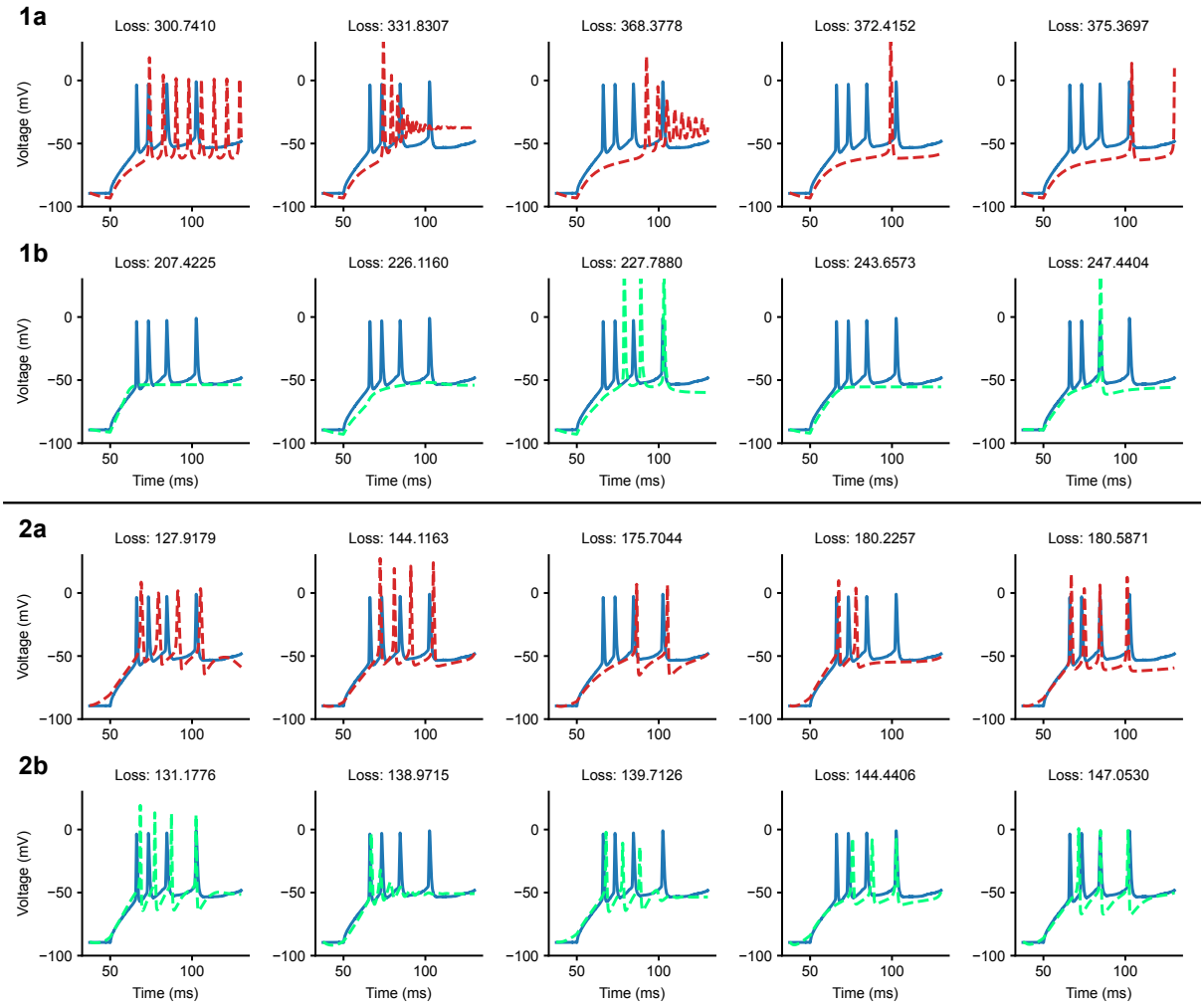

Figure 24: Same experimental setups as in Figure 23, now applied to cell 473601979 over 130 ms of x_o. Compared to cell 488683425, this neuron exhibits less regular spiking with four spikes and increasing inter-spike intervals, making it more challenging to fit. **Panel 1a:** Base model with standard channels. **Panel 1b:** Base model with learnable gating functions and exponent modifications. **Panel 2a:** Neural ODE-augmented model with standard channels. **Panel 2b:** Neural ODE-augmented model with learnable gating functions and exponent modifications.

Optimization proceeded in the same manner as in the previous experiments, with gradient-based parameter updates applied to both the standard parameters (e.g., conductance, reversal potential, length) as well as to the newly introduced parameters modifying the gating dynamics and the exponents. I evaluated the effect of these extensions on both short traces with one or two spikes and longer traces with multiple spikes, using the same cells as before for comparability.

The results show that introducing gating flexibility and learnable exponents generally improves fit quality across cells, though not in every case. In particular, the ability to slightly deform activation and inactivation curves (Figure 25) allows the models to better capture subthreshold dynamics, which is clear from the results of training the base models (panel 1 of Figure 23 and Figure 24). While these gains are smaller than those achieved through the neural ODE augmentation, they highlight the potential of structurally extending the biophysical model itself. This suggests that combining both strategies—refinements in gating dynamics and neural ODE augmentation—may provide the most powerful and interpretable approach to capturing the full range of neural dynamics.

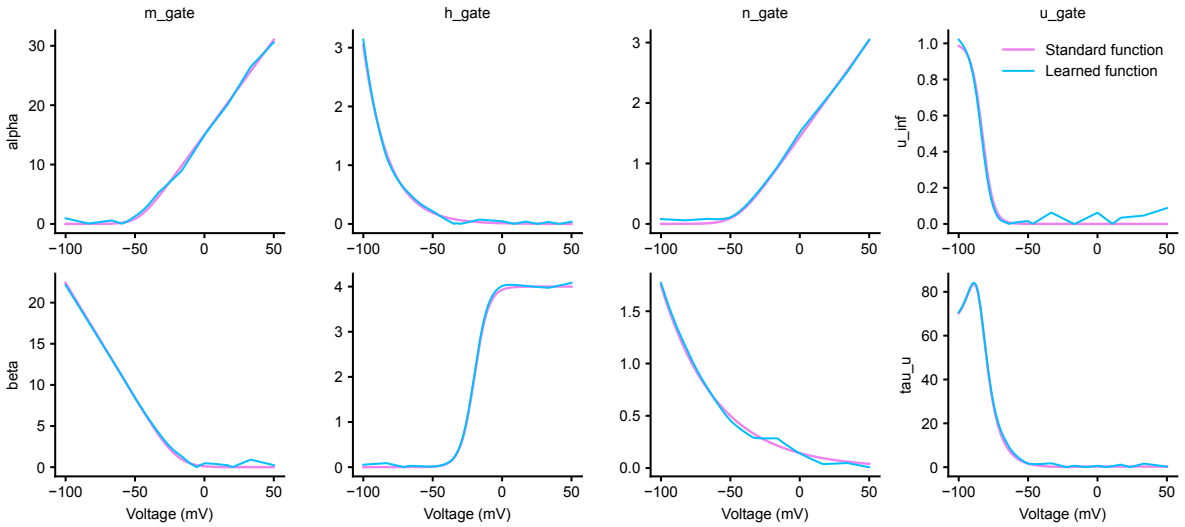

Figure 25: The learned activation and inactivation functions of the best-trained base model incorporating leak, Na, K, CaT, and Km channels, for cell 488683425.

7.5 Extrapolation to Longer Periods

To assess how well the learned models generalize beyond the training window, as a final experiment, I evaluated their extrapolation performance on the full period of data. Specifically, I compared models trained on shorter segments containing one–two spikes with those trained on longer segments containing three–four spikes, using the Pospischil-type channel set (leak, Na, K, CaT, and Km). Both the base models and their neural ODE-augmented counterparts with learnable gating functions and exponents were examined under this setup.

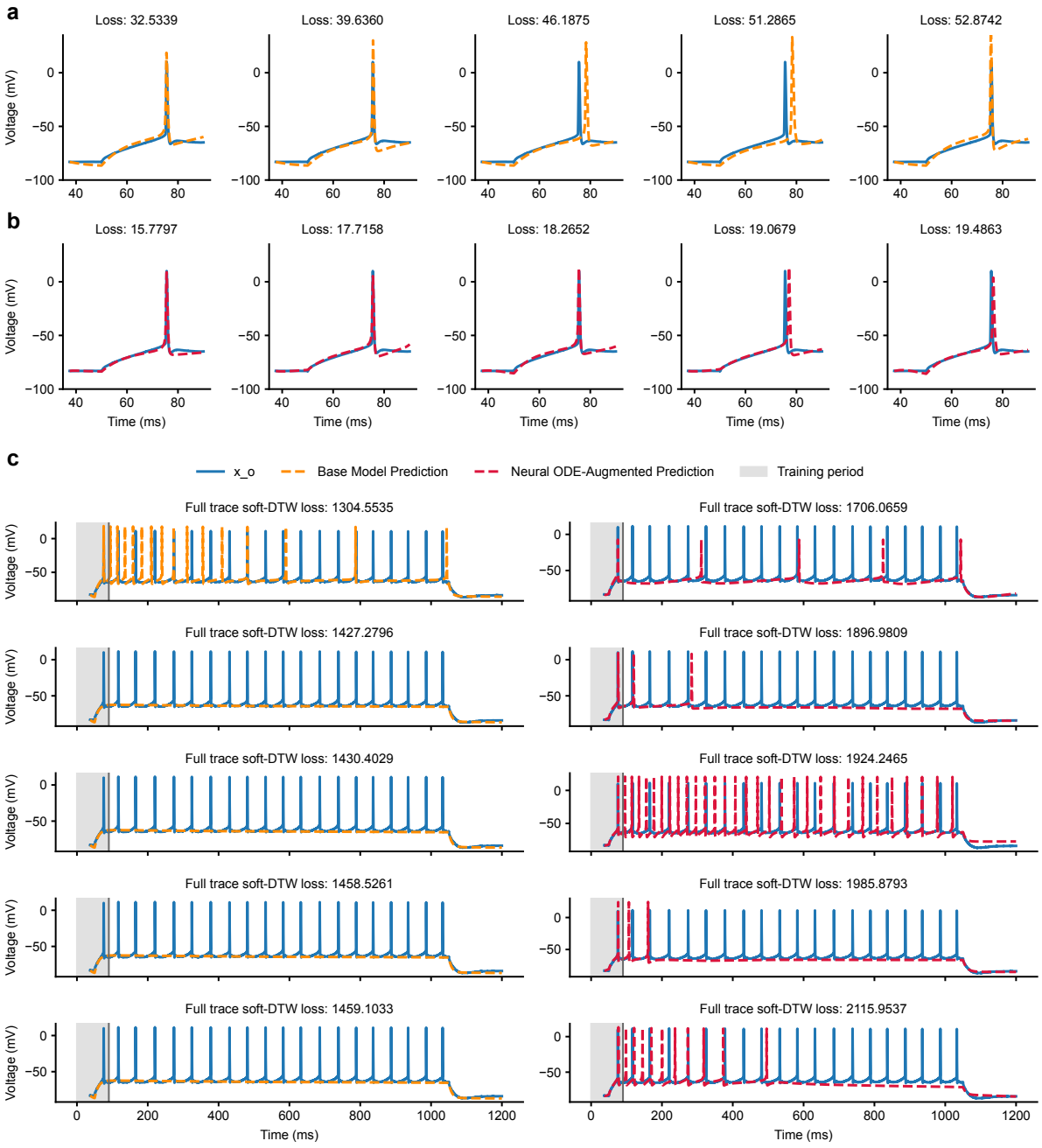

Figure 26: Training and extrapolation results for cell 488683425 using the base model (orange) and the neural ODE-augmented model (crimson). **Panel a:** The five best fits obtained by training the base model on 90 ms of x_o. **Panel b:** The five best fits obtained by training the neural ODE-augmented model with learnable gating functions and exponents on the same data segment. **Panel c:** The five best extrapolated fits of the trained base and augmented models to the full recording period.

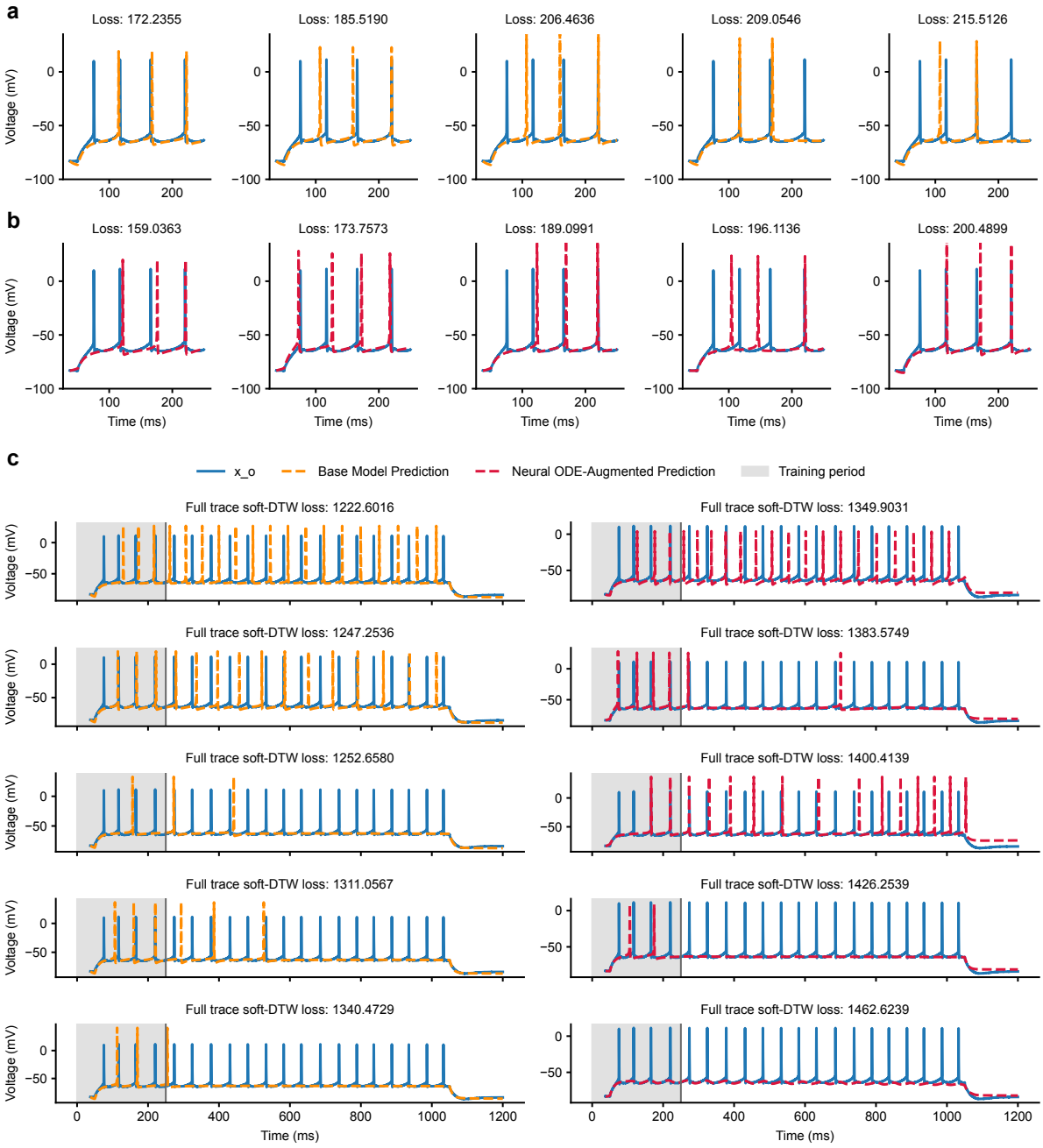

Figure 27: Training and extrapolation results for cell 488683425 using the base model (orange) and the neural ODE-augmented model (crimson). **Panel a:** The five best fits obtained by training the base model on 250 ms of x_o. **Panel b:** The five best fits obtained by training the neural ODE-augmented model with learnable gating functions and exponents on the same data segment. **Panel c:** The five best extrapolated fits of the trained base and augmented models to the full recording period.

The results show that neither extending the training window nor introducing neural ODE augmentation provides a consistent advantage for extrapolation (Figure 26 and Figure 27). While the neural ODE-augmented models consistently achieved lower training loss for all examined segment lengths (90 ms, 150 ms, 200 ms, and 250 ms), their extrapolated loss was typically higher than that of the corresponding base models. Similarly, training on longer segments did not reliably reduce the extrapolation error, regardless of whether the neural ODE augmentation was used.

The reasons for this discrepancy are not yet fully understood and will require further study. One likely factor is a form of overfitting: the neural ODE augmentation provides additional flexibility that allows the model to closely match the training data but can capture dynamics that fail to generalize outside

the training window. This effect may be linked to instabilities in the latent state u, which in some cases showed signs of divergence, although such behavior was not always evident. Another possibility is that the learned dynamics of the neural ODE augmentation interact with the channel mechanisms in ways that remain stable over the training segment but degrade when extrapolated.

These findings suggest that, while neural ODE augmentation consistently improves local fits within the training interval, its benefits do not straightforwardly extend to extrapolation. Likewise, simply increasing the training duration does not guarantee better generalization. Addressing these limitations may require explicit regularization of the latent dynamics or constraints designed to stabilize long-term behavior, potentially in combination with richer training data.

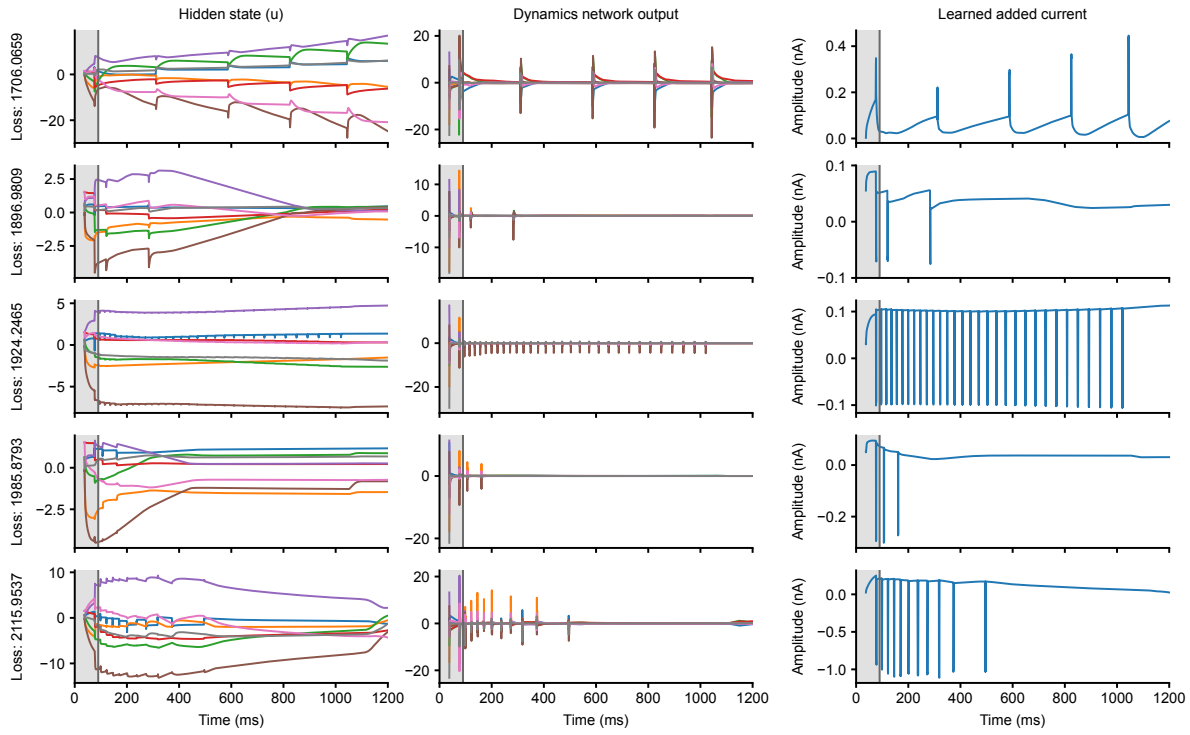

Figure 28: Evolution of the latent state u and the added current learned by the neural ODE from the five best extrapolated models after training over 90 ms of x_o.

8 Discussion

In this thesis, I addressed a key challenge in computational neuroscience: the limited expressivity of traditional biophysical neuron models. While mechanistic formulations such as Hodgkin–Huxley-type systems provide an interpretable link between ion channel properties and membrane dynamics, they remain notoriously difficult to fit to experimental recordings. This difficulty has two main sources that guided my work from the very beginning. First, due to the complexity of the underlying nonlinear dynamics, the neural voltage response is highly sensitive to model parameters; even small mismatches can lead to large deviations in voltage trajectories. Second, simplified model structures, such as the single-compartment approximation used here, discard features of neural dynamics that no amount of parameter tuning can recover.

To overcome these limitations, I explored the idea of augmenting such models with neural ordinary differential equations, which provide a flexible, data-driven mechanism that complements the interpretable biophysical backbone. Using this hybrid approach, I trained the biophysical parameters and the neural network weights and biases in parallel.

The main results of this work demonstrate that such a hybrid design can substantially improve local fits to both multi-compartment simulations and observed voltage recordings. Across multiple experimental setups, neural ODE augmentation consistently reduced training loss and enabled the models to reproduce fine-scale details of the traces that were inaccessible to the base models alone. To the best of my knowledge, this is the first systematic application of neural ODE augmentation to fitting conductance-based neuron models. This establishes a proof of principle: by combining interpretable parameters with a learnable current produced by the neural ODE, it is possible to push beyond the expressive limits of classical models while still retaining a biophysical grounding.

At the same time, this approach brings its own challenges. The added flexibility of the neural ODE comes at the cost of potential instability and overfitting, which affect performance both during training and in extrapolation scenarios. Early experiments showed that the latent state u can easily develop unstable dynamics, even in the simplest neural ODE cases. This observation echoes recent concerns in the literature about stability in neural ODE training and motivated me to explore careful initialization strategies, one of which I applied throughout my workflow. I also tested methods to address the pronounced differences in voltage responses between spiking and subthreshold regimes, which led me to adopt two separate neural networks for the two regimes in later experiments. Beyond the neural ODE component, I investigated modifying the biophysical model itself by using different sets of Pospischil-type channels. This proved beneficial, and it also allowed me to experiment with learnable gating functions and exponents in the channel dynamics.

In my final experiments, I found that while neural ODE–augmented models achieved the best local fits, their extrapolated performance outside the training window was often worse than that of the simpler base models. This discrepancy indicates a form of overfitting: the latent dynamics learned by the neural ODE capture fine-scale patterns in the training segment, but do not generalize robustly to unseen portions of the data.

Beyond the technical aspects, these findings contribute to a broader discussion about the role of hybrid modeling in computational neuroscience. Traditional conductance-based models offer mechanistic interpretability but struggle to reproduce the richness of experimental recordings, while purely data-driven approaches can achieve accurate fits at the expense of explanatory power. By combining the two, neural ODE augmentation illustrates a promising middle ground: interpretable parameters remain identifiable, yet the model retains sufficient flexibility to adapt to complex experimental traces.

At the same time, the limitations observed here highlight important considerations for future work. First, the issue of extrapolation underscores the need for additional inductive biases or regularization strategies that can guide the neural ODE toward capturing dynamics that generalize beyond the training segment. Second, the instability of the latent state suggests that architectural constraints or alternative formulations, such as stability-preserving recurrent networks or physics-informed neural ODEs, could improve robustness. Third, the distinction between spiking and subthreshold regimes points to the importance of regime-specific modeling choices, for example, explicitly constraining the additive current to act differently across these dynamical states.

Looking forward, several directions appear particularly promising. Applying this framework to larger multi-compartment models would test its ability to capture spatially distributed dynamics, while extending it to larger experimental datasets could assess its utility beyond the boundaries of my thesis.

Moreover, integrating probabilistic inference techniques could allow not only point estimation of parameters but also uncertainty quantification, which is critical for scientific interpretability. Finally, the general principle of augmenting mechanistic models with neural ODEs or other learnable components may extend well beyond neuroscience, offering a blueprint for hybrid modeling in areas of biology where mechanistic understanding is strong but incomplete.

Use of Artificial Intelligence Tools

All AI tools employed in this thesis, along with their version numbers and purpose, are listed here:

Tool	**Version**	**Use**
Writefull	Most recent version integrated in Overleaf (as of 01.10.2025)	Spelling and grammar correction
ChatGPT	GPT-5, GPT-3.5	Spelling and grammar correction Support with developing software
GitHub Copilot	GPT-4.1 Copilot	Support with developing software

References

Adams, P. R., Brown, D., and Constanti, A. (1982). Pharmacological inhibition of the m-current. *The Journal of Physiology*, 332(1):223–262.

Almog, M. and Korngreen, A. (2016). Is realistic neuronal modeling realistic? *Journal of Neurophysiology*, 116(5):2180–2209.

Bhalla, U. S. and Bower, J. M. (1993). Exploring parameter space in detailed single neuron models: simulations of the mitral and granule cells of the olfactory bulb. *Journal of Neurophysiology*, 69(6):1948–1965.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs.

Brette, R. (2015). What is the most realistic single-compartment model of spike initiation? *PLOS Computational Biology*, 11:1–13.

Brette, R. and Gerstner, W. (2005). Adaptive exponential integrate-and-fire model as an effective description of neuronal activity. *Journal of Neurophysiology*, 94(5):3637–3642.

Chen, R. T., Rubanova, Y., Bettencourt, J., and Duvenaud, D. K. (2018). Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31.

Colbert, C. M. and Pan, E. (2002). Ion channel properties underlying axonal action potential initiation in pyramidal neurons. *Nature Neuroscience*, 5(6):533–538.

Cuturi, M. and Blondel, M. (2018). Soft-DTW: a differentiable loss function for time-series.

Deistler, M., Kadhim, K. L., Pals, M., Beck, J., Huang, Z., Gloeckler, M., Lappalainen, J. K., Schröder, C., Berens, P., Gonçalves, P. J., and Macke, J. H. (2024). Differentiable simulation enables large-scale training of detailed biophysical models of neural dynamics. *bioRxiv*.

Destexhe, A., Bal, T., McCormick, D. A., and Sejnowski, T. J. (1996a). Ionic mechanisms underlying synchronized oscillations and propagating waves in a model of ferret thalamic slices. *Journal of Neurophysiology*, 76(3):2049–2070.

Destexhe, A., Contreras, D., Steriade, M., Sejnowski, T., and Huguenard, J. (1996b). In vivo, in vitro, and computational analysis of dendritic calcium currents in thalamic reticular neurons. *The Journal of Neuroscience*, 16(1):169–185.

Druckmann, S., Berger, T. K., Hill, S., Schürmann, F., Markram, H., and Segev, I. (2008). Evaluating automated parameter constraining procedures of neuron models by experimental and surrogate data. *Biological Cybernetics*, 99(4):371–379.

Dupont, E., Doucet, A., and Teh, Y. W. (2019). Augmented neural ODEs. *Advances in Neural Information Processing Systems*, 32.

Finlay, C., Jacobsen, J.-H., Nurbekyan, L., and Oberman, A. (2020). How to train your neural ODE: the world of jacobian and kinetic regularization. In *Proceedings of the 37th International Conference on Machine Learning*, pages 3154–3164.

Georgiev, D. D. (2004). The nervous principle: active versus passive electric processes in neurons. *Electroneurobiología*, 12(2):169–230.

Gerstner, W., Kistler, W. M., Naud, R., and Paninski, L. (2014). *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, Cambridge.

Goodman, D. F. and Brette, R. (2009). The Brian simulator. *Frontiers in Neuroscience*, 3:643.

Gutenkunst, R. N., Waterfall, J. J., Casey, F. P., Brown, K. S., Myers, C. R., and Sethna, J. P. (2007). Universally sloppy parameter sensitivities in systems biology models. *PLOS Computational Biology*, 3(10):1–8.

Hines, M. L. and Carnevale, N. T. (1997). The NEURON simulation environment. *Neural Computation*, 9(6):1179–1209.

Hodgkin, A. L. and Huxley, A. F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544.

Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.

Huh, D. and Sejnowski, T. J. (2018). Gradient descent for spiking neural networks. *Advances in Neural Information Processing Systems*, 31.

Huys, Q. J. M., Ahrens, M. B., and Paninski, L. (2006). Efficient estimation of detailed single-neuron models. *Journal of Neurophysiology*, 96(2):872–890.

Huys, Q. J. M. and Paninski, L. (2009). Smoothing of, and parameter estimation from, noisy biophysical recordings. *PLOS Computational Biology*, 5(5):1–16.

Jolivet, R., Schürmann, F., Berger, T. K., Naud, R., Gerstner, W., and Roth, A. (2008). The quantitative single-neuron modeling competition. *Biological Cybernetics*.

Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Köhler, M., Hirschberg, B., Bond, C., Kinzie, J. M., Marrion, N., Maylie, J., and Adelman, J. (1996). Small-conductance, calcium-activated potassium channels from mammalian brain. *Science*, 273(5282):1709–1714.

Kole, M. H., Hallermann, S., and Stuart, G. J. (2006). Single Ih channels in pyramidal neuron dendrites: properties, distribution, and impact on action potential output. *Journal of Neuroscience*, 26(6):1677–1687.

Lei, C. L. and Mirams, G. R. (2021). Neural network differential equations for ion channel modelling. *Frontiers in Physiology*, 12:708944.

Ljung, L. (1999). *System identification (2nd ed.): Theory for the user*. Prentice Hall PTR.

Lu, Y., Zhong, A., Li, Q., and Dong, B. (2018). Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *Proceedings of the 37th International Conference on Machine Learning*, pages 3276–3285.

Lueckmann, J.-M., Goncalves, P. J., Bassetto, G., Öcal, K., Nonnenmacher, M., and Macke, J. H. (2017). Flexible statistical inference for mechanistic models of neural dynamics. *Advances in neural Information Processing Systems*, 30.

Maclaurin, D., Duvenaud, D., and Adams, R. P. (2015). Autograd: Effortless gradients in Numpy. In *ICML 2015 AutoML workshop*, volume 238.

Marder, E. and Taylor, A. L. (2011). Multiple models to capture the variability in biological neurons and networks. *Nature Neuroscience*, 14(2):133–138.

Markram, H., Muller, E., Ramaswamy, S., Reimann, M. W., Abdellah, M., Sanchez, C. A., Ailamaki, A., Alonso-Nanclares, L., Antille, N., Arsever, S., et al. (2015). Reconstruction and simulation of neocortical microcircuitry. *Cell*, 163(2):456–492.

Nocedal, J. (1980). Updating quasi-newton matrices with limited storage. *Mathematics of Computation*, 35(151):773–782.

Nogaret, A. (2022). Approaches to parameter estimation from model neurons and biological neurons. *Algorithms*, 15(5):168.

Paninski, L., Pillow, J., and Lewi, J. (2007). Statistical models for neural encoding, decoding, and optimal stimulus design. *Progress in Brain Research*, 165:493–507.

Pontryagin, L. S. (2018). *Mathematical theory of optimal processes*. Routledge.

Pospischil, M., Toledo-Rodriguez, M., Monier, C., Piwkowska, Z., Bal, T., Frégnac, Y., Markram, H., and Destexhe, A. (2008). Minimal Hodgkin-Huxley type models for different classes of cortical and thalamic neurons. *Biological Cybernetics*, 99:427–441.

Rackauckas, C., Ma, Y., Martensen, J., Warner, C., Zubov, K., Supekar, R., Skinner, D., Ramadhan, A., and Edelman, A. (2020). Universal differential equations for scientific machine learning. *arXiv preprint arXiv:2001.04385*.

Rall, W. (1962). Electrophysiology of a dendritic neuron model. *Biophysical Journal*, 2(2 Pt 2):145.

Rattay, F., Greenberg, R. J., and Resatz, S. (2002). Neuron modeling. *Handbook of Neuroprosthetic Methods*, pages 39–71.

Reuveni, I., Friedman, A., Amitai, Y., and Gutnick, M. J. (1993). Stepwise repolarization from Ca^{2+} plateaus in neocortical pyramidal cells: Evidence for nonhomogeneous distribution of HVA Ca^{2+} channels in dendrites. *Journal of Neuroscience*, 13(11):4609–4621.

Rossum, M. C. W. v. (2001). A novel spike distance. *Neural Computation*, 13(4):751–763.

Rubanova, Y., Chen, R. T., and Duvenaud, D. K. (2019). Latent ordinary differential equations for irregularly-sampled time series. *Advances in Neural Information Processing Systems*, 32.

Ruthotto, L. and Haber, E. (2020). Deep neural networks motivated by partial differential equations. *Journal of Mathematical Imaging and Vision*, 62(3):352–364.

Sakoe, H. and Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49.

Traub, R. D. and Miles, R. (1991). *Neuronal Networks of the Hippocampus*. Cambridge University Press.

Van Geit, W., Achard, P., and De Schutter, E. (2007). Neurofitter: A parameter tuning package for a wide range of electrophysiological neuron models. *Frontiers in Neuroinformatics*, 1:89.

Van Geit, W., Gevaert, M., Chindemi, G., Rössert, C., Courcol, J.-D., Muller, E. B., Schürmann, F., Segev, I., and Markram, H. (2016). Bluepyopt: Leveraging open source software and cloud infrastructure to optimise model parameters in neuroscience. *Frontiers in Neuroinformatics*, 10.

Victor, J. D. and Purpura, K. P. (1997). Metric-space analysis of spike trains: theory, algorithms and application. *Network: Computation in Neural Systems*, 8(2):127–164.

Werbos, P. J. (2002). Backpropagation through time: what it does and how to do it. In *Proceedings of the IEEE*, volume 78, pages 1550–1560.

Westny, T., Mohammadi, A., Jung, D., and Frisk, E. (2024). Stability-informed initialization of neural ordinary differential equations. In *Proceedings of the 41st International Conference on Machine Learning*.

White, A., Kilbertus, N., Gelbrecht, M., and Boers, N. (2023). Stabilized neural differential equations for learning dynamics with explicit constraints. *Advances in Neural Information Processing Systems*, 36:12929–12950.

Yamada, W. M., Koch, C., and Adams, P. R. (1989). Multiple channels and calcium dynamics. *Methods in neuronal modeling*, 4:97–133.

Yang, S., Deng, B., Wang, J., Li, H., Lu, M., Che, Y., Wei, X., and Loparo, K. A. (2020). Scalable digital neuromorphic architecture for large-scale biophysically meaningful neural network with multi-compartment neurons. *IEEE Transactions on Neural Networks and Learning Systems*, 31(1):148–162.